

CHAPTER 1

INTRODUCTION

1.1 Introduction

Every year, thousands of people - especially women and college students - find themselves where they need urgent help but have no easy, trusted way to raise an alarm. A shout or a phone call is often not possible. RakshaSafe is a centralized, web-based emergency response system built to close this gap: a user raises an SOS in one tap, chosen contacts are alerted by SMS and e-mail, and responders reach the incident with live location, a computed risk level and nearby resources.

In an emergency the delay between 'something is wrong' and 'someone trustworthy can act' is too long. RakshaSafe attacks that delay with a panic button, an intelligent risk score and a complete responder workflow across three layers: React frontend, Node.js/Express backend and a Python AI risk service.

1.2 Background

This project started as a final-year B.Sc. (Information Technology) requirement, but the motivation is real-world. Panic-button concepts are old; what changed is that nearly every student now carries a smartphone with GPS, mobile data and messaging apps. An emergency system no longer needs dedicated hardware - the phone itself is the panic button.

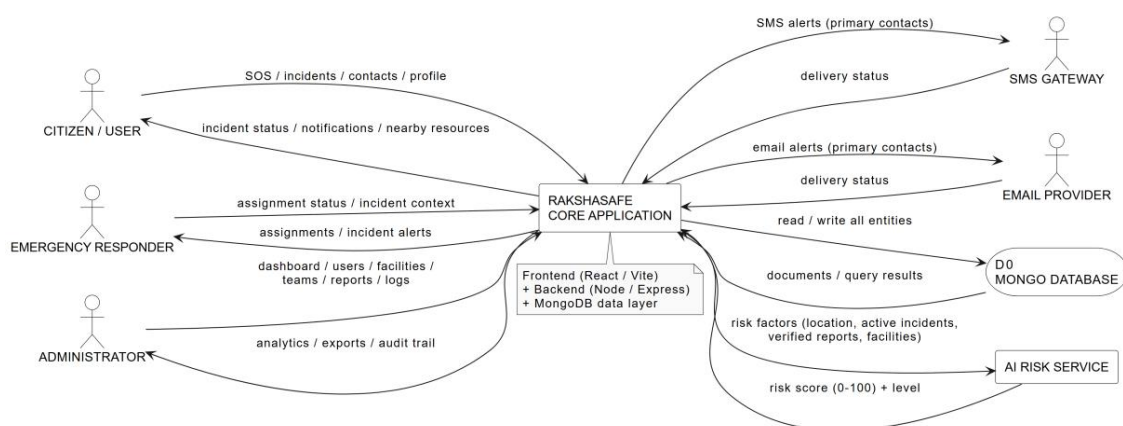


Figure 1.1: System context - who talks to RakshaSafe (DFD Level 0).

1.3 Objectives of the Project

- To design and implement a location-aware emergency reporting system usable by non-technical users under stress.
- To notify primary emergency contacts automatically over SMS and e-mail.
- To produce a meaningful, explainable risk score (0-100) for every incident.
- To give rescue teams and administrators a complete assignment and management workflow with full auditability.
- To verify the whole system with real automated and manual tests - 127 unit, 67 integration and 70 live system checks (264 in total).

1.4 Purpose

This document is the complete project report. It records what RakshaSafe must do (Chapter 3), how it is designed (Chapter 4), how it was built (Chapter 5), how it was tested and with what results (Chapter 6), and what the project concludes (Chapter 7). An examiner can follow the journey from problem to a tested, working system.

1.5 Scope

1.5.1 In Scope

- User accounts, profiles, SOS incidents and the full incident lifecycle.
- Emergency contacts with SMS / e-mail notification and an in-app feed.
- Facilities, rescue teams, assignments and unsafe-area reports.
- Admin dashboard, report generation and a complete audit log.
- AI risk scoring with a documented deterministic model.

1.5.2 Out of Scope

- Native Android / iOS apps (a responsive web UI is provided instead).
- Procurement or integration with government emergency call centres.
- Training of responders or change management inside a real organisation.

1.6 Applicability

Primary users are students travelling to and from campus, office-goers in the late evening and residents of areas with weak formal emergency coverage. Secondary users are college security teams, NGO volunteers and local responders who want a structured way to receive, acknowledge and resolve alerts. A live production SMS gateway is expensive, so every provider is optional; the design records exactly what happened, including NOT_CONFIGURED outcomes.

1.7 Achievements of the Project

- A working, tested emergency response system with one-tap SOS and live location capture.
- Automatic SMS / e-mail alerts to primary emergency contacts, with honest delivery status recording, including NOT_CONFIGURED outcomes.
- A deterministic, explainable AI risk score (0-100) computed for every incident.
- A complete responder workflow: rescue-team assignment, status progression and notifications.
- An administrator dashboard with user, facility, team and report management plus an audit log.
- Executed test evidence: 264 checks (127 unit, 67 integration, 70 live system), all passing.

1.8 Organization of the Report

Chapter 2 surveys existing systems and technologies and records the survey exactly as it was prepared. Chapter 3 analyses the requirements - functional and non-functional, data, hardware, software and the project schedule. Chapter 4 presents the system design: architecture, modules, database collections, API routes, risk model, user interfaces and security. Chapter 5 implements that design - it lists the main parts of the real code with screenshots of the running website. Chapter 6 tests the implemented system and records the real results for every module. Chapter 7 concludes the project and lists its limitations and future scope.

[illegible]

Safety at Your Fingertips: The History of and Case for Panic Buttons Title: Safety at Your Fingertips: The History of and Case for Panic Buttons Author: CM3 Building Solutions — Editorial Team

This reference explains how panic buttons have developed from simple physical alarms to modern digital safety solutions. A panic button provides a quick way for a person to indicate that an emergency has occurred.

Cell Phone Personal Safety Alarm Title: Cell Phone Personal Safety Alarm Author: Erik N. Hagenstad

Publication Year: 2016

Publisher/Source: United States Patent and Trademark Office (USPTO), U.S. Patent No. 9,451,061

Summary:

This reference describes a mobile safety system that combines different emergency features such as a panic button, warning functions, communication and location support.

The concept is related to Raksha because the project also tries to bring important emergency functions into one system. In Raksha, a user can create an emergency incident and the system can store the related incident and location information for further processing.

Multifunctional Personal Emergency Safety Device Title: Multifunctional Personal Emergency Safety Device Author: Jason L. Lassiter

Publication Year: 2017

Publisher/Source: United States Patent and Trademark Office (USPTO), U.S. Patent No. 9,697,720

Summary:

This reference describes an emergency device where one emergency action can start several safety-related functions.

The idea is useful for Raksha because an emergency report can involve different types of information. The system maintains incident details, user information, location, emergency contacts, notifications and response-related records in an organized way.

Global Positioning System (GPS) Title: Global Positioning System (GPS) Author: Multiple contributors Publication Year: 2026

Publisher/Source: Wikipedia, The Free Encyclopedia

Summary:

GPS and other location technologies are used to determine the geographical position of a device. Location information is especially useful in emergency situations because it helps identify where an incident has been reported.

Raksha uses the available location information from the user's device through the Browser/Device Geolocation API. The system can store latitude and longitude along with other available location details. The accuracy of the location depends on device capability, permissions, network conditions and available location services.

Brief History of GPS Title: Brief History of GPS

Author: The Aerospace Corporation

Publication Year: Not specified in the supplied reference

Publisher/Source: aerospace.org

Summary:

This reference explains the development of satellite-based navigation and location technology. Location services have become a common part of modern web and mobile applications.

For Raksha, location information is useful for connecting an emergency incident with a geographical position. The available latitude and longitude can be stored and later displayed or used by the administrator while managing incidents.

Who Invented GPS? Powerful Minds Behind the History of GPS Title: Who Invented GPS? Powerful Minds Behind the History of GPS Author: History Cooperative — Editorial Team

Publication Year: 2023 Publisher/Source: historycooperative.org Summary:

This reference provides a historical overview of the researchers and developments that contributed to GPS technology.

The study of GPS is relevant to Raksha because location is an important part of emergency management. When a user reports an incident, the system can associate the available location information with that incident and make it easier for the administrator to understand the reported location.

WhatsApp Title: WhatsApp

Author: Jan Koum and Brian Acton — founders; source is Wikipedia

Publication Year: 2026

Publisher/Source: Wikipedia, The Free Encyclopedia

Summary:

WhatsApp is a commonly used communication platform that allows users to exchange messages and other information.

WhatsApp was studied because messaging applications can be useful for emergency communication. In Raksha, WhatsApp-based communication can be supported through a configured provider where available. It is treated as a communication option rather than assuming that an external WhatsApp service will always be available.

The History of WhatsApp Business API Title: The History of WhatsApp Business API Author: Qiscus — Editorial Team Publication Year: 2026

Publisher/Source: qiscus.com

Summary:

The WhatsApp Business API provides a way for organizations and software systems to communicate with users through automated messages.

This technology was studied because an emergency management system may require communication with users or contacts. In Raksha, WhatsApp communication can be connected through a configured communication provider if the required service is available and properly configured.

Twilio Title: Twilio

Author: Jeff Lawson, Evan Cooke and John Wolthuis

Publication Year: 2026

Publisher/Source: Wikipedia, The Free Encyclopedia

Summary:

Twilio is a cloud communication platform that provides programmable communication services such as SMS and voice.

This technology was surveyed because emergency systems may require external communication services. Raksha supports the concept of configurable communication providers for services such as email, SMS or WhatsApp, depending on the services configured for the project.

How Jeff Lawson Founded Twilio: Build with Conviction or Risk Burnout Title: How Jeff Lawson Founded Twilio: Build with Conviction or Risk Burnout Author: Kyle Kelly-Yahner

Publication Year: 2015

Publisher/Source: Twilio Inc. — Company Blog

Summary:

This reference discusses the development of Twilio and the use of APIs to provide communication services to software applications.

The main idea is useful for Raksha because APIs can connect a web application with external services. Raksha uses a REST API-based backend, while external communication providers can be configured when required.

Evolution of Speech Recognition Technology — From Audrey to Siri Title: Evolution of Speech Recognition Technology — From Audrey to Siri Author: Verbit — Editorial Team

Publication Year: 2019

Publisher/Source: verbit.ai

Summary:

This reference explains how speech recognition developed from early systems into modern voice-based applications.

Speech recognition was studied because voice-based emergency interaction can be useful in situations where a person may not be able to operate a device normally. It can be considered as an additional feature for future versions of Raksha.

Nervous System: Audrey and the Dragon — The History of Voice Recognition Technology

Nervous System: Audrey and the Dragon — The History of Voice Recognition Technology

Author: David Kalat

Publication Year: 2021

Publisher/Source: Berkeley Research Group (BRG) Insights

Summary:

This reference describes the early development of speech recognition and how computers gradually became better at understanding spoken words.

The study is relevant to Raksha because voice-based emergency interaction can provide an alternative method of communication. However, it is considered an optional enhancement and is not treated as a required part of the core web application

Google Translate Title: Google Translate

Author: Franz Josef Och — associated with the original Google Translate architecture

Publication Year: 2026

Publisher/Source: Wikipedia, The Free Encyclopedia

Summary:

Translation technology allows information to be converted from one language into another. This can be useful in emergency situations where people may communicate in different languages.

The technology was studied for Raksha because multilingual support can make a safety system easier to use. Automated translation through an external service can be added as an enhancement when the required service is configured.

Celebrating 20 Years of Google Translate: Fun Facts, Tips and New Features to Try Title: Celebrating 20 Years of Google Translate: Fun Facts, Tips and New Features to Try

Author: Rose Yao

Publication Year: 2026

Publisher/Source: Google Blog / The Keyword

Summary:

This reference explains the development of Google Translate and its support for different languages.

Multilingual communication can be useful in disaster management because users may prefer to communicate in their local language. Therefore, translation technology was studied as a possible future enhancement for Raksha.

Decision Tree and Random Forest Algorithms: Decision Drivers Title: Decision Tree and Random Forest Algorithms: Decision Drivers Author: History of Data Science — Editorial Team

Publication Year: 2022

Publisher/Source: historyofdatascience.com

Summary:

This reference explains Decision Tree and Random Forest algorithms and their use in classification problems.

Machine learning can be useful in emergency applications for analysing information and identifying possible incorrect or accidental alerts. The concept was studied for Raksha as a possible way to improve future intelligent alert analysis.

Random Forests Title: Random Forests Author: Leo Breiman Publication Year: 2001

Publisher/Source: Machine Learning Journal, Springer

Summary:

Leo Breiman's research introduced the Random Forest method, which combines multiple decision trees for classification.

This research provides a useful academic background for understanding machine-learning-based classification. For Raksha, such techniques may be used in future development for analysing emergency data or reducing false alerts, depending on the final implementation.

MongoDB and Mongoose Technology: MongoDB

Database Type: NoSQL Document Database

ODM: Mongoose Summary:

MongoDB is a document-based database used for storing application data. Mongoose works with MongoDB in Node.js applications and helps define schemas, validate data and manage database operations.

MongoDB is suitable for Raksha because the system contains different types of information such as users, emergency contacts, incidents, locations, notifications, rescue teams, rescue assignments, facilities, risk information and reports.

The Raksha database is organized into separate collections. References between related documents are maintained at the application level using Mongoose. This makes the database suitable for managing the different modules of the system.

Node.js and Express.js Technology: Node.js Framework: Express.js

API Architecture: REST API

Summary:

Node.js provides the runtime environment for the backend of a web application. Express.js can be used with Node.js to create APIs and handle requests from the frontend.

Raksha uses Node.js and Express.js as its backend technologies. The backend manages application requests, communicates with MongoDB, handles user authentication and supports different system modules through REST APIs.

This technology is important because the frontend and database need a backend layer through which data can be processed securely and consistently. The original survey also identified Node.js and Express.js as suitable backend technologies for Raksha.

React.js, TypeScript, Vite and Tailwind CSS Frontend: React.js

Programming Language: TypeScript

Build Tool: Vite Styling: Tailwind CSS Summary:

React.js is used for developing interactive web interfaces using reusable components. TypeScript adds type checking and helps make the frontend code easier to maintain. Vite provides a fast development and build environment, while Tailwind CSS is used for designing the user interface.

These technologies are suitable for Raksha because the system contains multiple screens for users and administrators. These include login, user profile, SOS/incident management, emergency contacts, emergency resources, notifications, risk information, admin dashboard and reports.

The use of React.js makes Raksha a web-based application, rather than a React Native mobile application. This is consistent with the software requirements of the project.

Python, FastAPI and AI Service Integration AI Backend: Python

AI Framework: FastAPI

AI Integration: Configured AI Service API

Summary:

Python is widely used for developing data-processing and artificial-intelligence-related applications. FastAPI provides a framework for creating APIs using Python.

These technologies are relevant to Raksha because the project includes an AI-related backend/service layer in its software requirements. A configured AI service can be connected to the main backend through an API when required.

The AI component is kept separate from the main Node.js backend so that AI-related processing can be managed independently. This structure also makes it easier to modify or extend the AI functionality in the future.

OpenStreetMap, Leaflet and Location-Based Services Mapping: OpenStreetMap

Map Library: Leaflet.js

Location: Browser/Device Geolocation API Map Service: Configured Map Service Summary:

OpenStreetMap provides geographical map data, while Leaflet.js is a lightweight JavaScript library used to display interactive maps.

These technologies are useful for Raksha because location is an important part of emergency and disaster management. The system can use available device location information and display relevant incident or risk-related locations on the administrative interface.

Leaflet can also help display location-based information such as incident locations and risk zones in a simple visual form. OpenStreetMap can be used as the map-data source where configured. The original technology survey also identified OpenStreetMap and Leaflet as useful technologies for the Raksha dashboard.

Summary: Raksha is a web-based Women Safety and Disaster Management System. It uses React.js, TypeScript, Vite and Tailwind CSS for the frontend, Node.js and Express.js for the backend, and MongoDB with Mongoose for data storage. The system uses location services and map technologies for emergency location management. JWT and bcrypt support authentication and password security. Python and FastAPI are used for the AI-related service layer, while communication and other advanced technologies can be configured when required. The technology survey helped in selecting suitable technologies for developing and managing the Raksha system.

CHAPTER 3

REQUIREMENT AND ANALYSIS

3.1 Introduction

Requirement analysis defines who the users are, what functions the system must perform, and the rules it must obey. This chapter presents the authorised requirement input for RakshaSafe: functional and non-functional requirements, data, hardware and software needs, and the project plan, with its diagrams and images.

3.1 Requirement Analysis

Requirement analysis defines the users, functions, resources and requirements of the Raksha system. It supports SOS requests, incident and location information, emergency resources, admin management and centralized data storage. It also covers functional, non-functional, hardware, software and database requirements.

3.2 Problem Definition

Problem Statement

Users may face emergencies such as harassment, accidents, medical emergencies, fire, floods and earthquakes where quick and organized information is required. Administrators may also find it difficult to monitor incidents and maintain updated emergency resources.

Main Problems:

Lack of a centralized safety platform.

Difficulty recording and tracking incidents.

Difficulty finding emergency facilities and resources.

Lack of organized incident history.

Difficulty managing emergency resources.

Lack of centralized admin monitoring.

Risk of unauthorized access to sensitive data.

Difficulty maintaining unsafe-area and risk information.

Difficulty generating useful reports and statistics.

3.2.1 Lack of Centralized Safety Records

Emergency incidents may contain important information such as incident type, description, user information, location, priority and current status.

If this information is not stored systematically, it becomes difficult to retrieve, monitor and analyse later.

Raksha provides a centralized database where authorized application components can store and retrieve emergency incident information.

3.2.2 Difficulty in Sharing Location Information

Location helps identify where an incident occurred. Raksha can request the user's location and store available latitude and longitude with the incident. Location accuracy depends on device, GPS, network and permission availability. If unavailable, the system does not assume a false location.

3.2.3 Difficulty in Finding Emergency Resources

During an emergency, users may need quick information about hospitals, shelters, police stations, fire stations and relief centres. Raksha keeps these facility details organized, including name, type, address, contact number, location and operational status. Capacity and operating hours can also be maintained where available. The system separately stores information about registered response teams

3.2.4 Difficulty in Monitoring Emergency Incidents

Administrators require a structured method for monitoring emergency incidents.

Raksha provides an administrative dashboard where authorized administrators can:

View emergency incidents

View incident details

View available location information

Check incident priority

Check current status

Assign registered response resources

Update incident status

View incident history

Monitor system activity

The assignment and status features represent software-based coordination and record management. They do not guarantee physical response from external organizations.

3.2.5 Difficulty in Maintaining Incident History

Emergency incidents may require a chronological record for future reference, reporting and analysis.

Raksha maintains incident updates that record important activities such as:

Incident creation

Status change

Assignment

Progress update

Resolution

Closure

Cancellation

Administrative comments

3.2.6 Difficulty in Managing Emergency Resources

Hospitals, shelters, police stations, fire stations, relief centres and response teams may have different information.

Raksha therefore separates emergency resources into structured database records. Facilities such as hospitals and shelters are maintained through the Facilities collection, while registered response teams are maintained through the Rescue Teams collection. Administrators can add, edit, activate or deactivate resource information according to their permissions. The application displays stored resource information and does not assume real-time availability unless such information is actually connected to and maintained by a reliable source.

4.1.11 Notification Management

Notifications keep users informed about important system events such as incident creation, acknowledgement, assignment, status change, resolution, cancellation, and system updates. Supported channels may include In-app, Email, SMS, and WhatsApp, depending on configured services.

Notification records store Incident ID, Contact ID, Channel, Status, Provider Response, Attempt Count, Last Attempt Time, and Created Date.

Possible statuses are QUEUED, SENT, DELIVERED, FAILED, NOT_CONFIGURED, and UNAVAILABLE.

3.3 Proposed Solution

Raksha provides a centralized web application that allows registered users to create emergency incidents and access emergency-resource information.

When an SOS request is submitted, the system creates an incident record and stores the available incident details in the database. If location permission is granted, available location information is associated with the incident.

Administrators can monitor incidents through the administrative dashboard, update incident status, manage facilities and response-team information, and maintain administrative records.

The system can also provide notification functionality through configured communication services. Notification delivery depends on the availability and configuration of the selected provider.

The system may use a Python-based risk assessment service to process available incident and location-related factors and provide a risk score and risk level. The risk assessment is intended as software-based decision support and does not replace professional emergency assessment.

System Limitations

Raksha does not claim to provide

Automatic police FIR registration

Guaranteed police response

Guaranteed ambulance response

Guaranteed rescue-team response

Exact rescue ETA

Guaranteed 24×7 emergency response

Real-time hospital bed availability without a connected source

Offline emergency communication unless specifically implemented

Actual emergency response depends on authorized external organizations, service providers, network connectivity and real-world availability.

3.4 Requirements Specification

Requirements specification defines the functions and quality requirements of Raksha.

The requirements are divided into:

Functional Requirements

Non-Functional Requirements

User Requirements

System Constraints

Hardware Requirements

Software Requirements

Database Requirements

3.4.1 Functional Requirements

The major functional requirements of Raksha are:

User registration and login

User authentication

Role-based authorization

User profile management

Emergency contact management

SOS and incident creation

Incident information management

Location processing

Emergency facility management

Rescue-team management

Rescue assignment management

Incident status management

Notification management

Incident history management

Unsafe-area reporting

Risk-zone management

Risk assessment

Administrative dashboard

Analytics and reporting

Resource search and filtering

Administrative activity logging

Secure access to protected information

3.4.1.1 User Authentication and Authorization

The system shall provide authentication so that only registered users can access protected features.

Requirements:

The system shall allow users to register.

The system shall allow users to log in.

The system shall validate login credentials.

Passwords shall not be stored in plain text.

Passwords shall be securely hashed.

The system shall identify the role of the logged-in account.

Unauthorized users shall not access administrator functions.

Users shall be able to log out.

Protected API operations shall require proper authorization.

Authentication shall use secure token-based access.

3.4.1.2 Role-Based Access Control

Raksha separates system operations according to user roles.

User

A normal user can:

Register

Login

Logout

Manage profile

Manage emergency contacts

Create emergency incidents

Share location where available

View own incidents

View incident status

View notifications

View emergency resources

Submit unsafe-area reports

View relevant risk information where provided

Administrator

An administrator can:

Login securely

View emergency incidents

View incident details

View available locations

Update incident status

Manage facilities

Manage rescue-team records

Manage rescue assignments

Review unsafe-area reports

Manage risk-zone information

View dashboard statistics

View reports

Monitor administrative activity

3.4.1.3 User Profile Management

The system shall provide basic profile functionality.

Users may:

View profile information

Update permitted profile information

Manage account information

View emergency history

Manage emergency contacts

Logout from the application

Only required information should be collected.

3.4.1.4 Emergency Contact Management

The system shall allow users to maintain emergency contact information.

An emergency contact may contain:

Contact name

Phone number

Email address

Relationship

SMS notification preference

Email notification preference

Emergency contact information shall only be accessible to authorized users and system functions.

3.4.1.5 SOS / Emergency Incident Management

SOS management is one of the main functions of Raksha.

When a user submits an SOS or emergency request, the application creates an incident record.

An incident can contain:

Incident ID

User ID

Incident type

Category

Description

Priority

Location ID

Current status

Creation date and time

Resolution information, where applicable

Incident Types

Examples include:

Women Safety

Medical Emergency

Accident

Fire

Flood

Earthquake

Other Emergency

Incident Status

The system uses the following detailed workflow:

REPORTED → ACKNOWLEDGED → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED

An incident may also be marked:

CANCELLED

where applicable.

In the database design, the SOS or emergency request is stored as an Incident record.

3.4.1.6 Location Processing

The system can request location permission from the user's browser or device.

When available, the system may capture:

Latitude

Longitude

Address or location label

City

State

Country

Location accuracy

Location capture time

The system shall:

Request permission where required.

Capture coordinates when available.

Associate location data with the relevant incident.

Handle unavailable location data.

Avoid creating false location information.

Restrict access to sensitive location information.

Location accuracy depends on the user's device, browser permissions, GPS/network conditions and configured location services.

3.4.1.7 Emergency Facility Management

Raksha maintains information about emergency facilities.

The Facilities collection may contain:

Facility name

Facility type

Location ID

Phone number

Capacity, where maintained

Operational status

Operating hours

Created date

Updated date

Facility types may include:

Hospital

Shelter

Police Station

Fire Station

Relief Centre

The system does not claim real-time bed availability or actual operational response unless such information is connected to a reliable source.

3.4.1.8 Rescue Team Management

The system can maintain records of registered response teams.

A rescue-team record may contain:

Team name

Team type

Contact information

Email

Active status

Service area

Specialization

Team types may include:

Police

Medical

Fire

Volunteer

NGO

The system manages these records as software information and does not guarantee real-world team availability.

3.4.1.9 Rescue Assignment Management

Administrators can create a software-based assignment record between an incident and a registered response team.

The assignment may contain:

Incident ID

Team ID

Assigned by

Assigned date and time

Assignment status

Notes

Assignment status may include:

ASSIGNED

EN_ROUTE

ON_SCENE

COMPLETED

CANCELLED

These statuses represent information recorded within the application and should not be interpreted as guaranteed physical movement or response.

3.4.1.10 Incident Status and Monitoring

Administrators can monitor emergency incidents through the dashboard.

The dashboard may display:

Total incidents

Reported incidents

Acknowledged incidents

Assigned incidents

In-progress incidents

Resolved incidents

Closed incidents

Cancelled incidents

Recent incidents

Incident categories

Priority information

When an administrator changes an incident status, the system stores the change and can create an incident-update record and notification.

3. 4.1.11 Notification Management

Notifications help users stay updated about important events in the system. They may be sent when an emergency is created, acknowledged, assigned, updated, resolved, or cancelled.

Depending on available services, notifications can be sent through- In-app notifications- Email, SMS, WhatsApp

The system can store basic details such as incident ID, contact, channel, status, attempts, and date. Notification status may be QUEUED, SENT, DELIVERED, FAILED, NOT_CONFIGURED, or UNAVAILABLE.

3.4.1.12 Incident History

Raksha maintains a chronological activity record for incidents.

The history can contain:

Incident creation

Status change

Assignment

Progress update

Resolution

Closure

Cancellation

Administrative comment

The Incident Updates collection is used for maintaining this chronological history.

3.4.1.13 Unsafe Area Reporting

Users can report locations that they consider unsafe or risky.

An unsafe-area report may contain:

Reporter ID

Location ID

Category

Description

Severity

Verification status

Created date

Updated date

Administrators can review and verify submitted reports.

A user-submitted report should not automatically be treated as a confirmed dangerous location until it has been reviewed according to the implemented workflow.

3.4.1.14 Risk Zone Management

The system can maintain structured information about identified risk zones.

A risk-zone record may contain:

Zone name

Risk level

Geographic boundary

Risk factors

Last assessment date

Active status

Risk-zone information shall be managed according to administrator permissions.

3.4.1.15 Risk Assessment

Raksha can use a Python and Fast API-based risk assessment service to process available information related to an incident or location.

The service may produce:

Risk score

Risk level

Model version

Input factors

Assessment time

The risk score may use a range such as 0–100.

Risk levels may include:

LOW

MEDIUM

HIGH

CRITICAL

Risk assessment is intended to support information analysis. It does not provide guaranteed predictions or replace professional emergency decisions.

3.4.1.16 Disaster Category Management

The system can maintain structured disaster and emergency categories.

A category may contain:

Category name

Category code

Description

Default priority

Response-team requirement

Active status

This allows incident categories to be maintained without modifying the complete application structure.

3.4.1.17 Administrative Dashboard

The administrative dashboard provides a central management interface.

It can display:

Total users

Total incidents

Reported incidents

Active incidents

Resolved incidents

Cancelled incidents

Available facilities

Registered rescue teams

Recent activities

Notifications

Basic statistics

Dashboard information shall be generated from actual stored system data.

3.4.1.18 Analytics and Reporting

The system can generate basic analytics from stored data.

Examples include:

Number of incidents

Incidents by type

Incidents by priority

Incidents by status

Monthly incident count

Number of facilities

Number of shelters

Number of registered response teams

Unsafe-area reports

Risk-level distribution

Analytics represent stored application data and should not be presented as guaranteed predictions of future emergencies.

3.4.1.19 Resource Search

Users can search and filter emergency resources.

Filtering may be based on:

Resource type

Facility name

Area

Location

Service category

Operational status

Where a configured map service is available, location-based information may also be displayed through a map interface.

3.4.1.20 Administrative Activity Logging

Administrative actions should be recorded for accountability.

An administrative log may contain:

Administrator ID

Action

Target type

Target ID

Action details

IP address, where collected

Date and time

Administrative logs should be accessible only to authorized administrators.

3.4.1.21 Data Management

The system shall maintain structured records for:

Users

Emergency Contacts

Incidents

Locations

Notifications

Rescue Teams

Rescue Assignments

Incident Updates

Unsafe Area Reports

Risk Zones

Risk Assessments

Facilities

Disaster Categories

Administrative Logs

Reports

Database operations shall be performed through authorized backend APIs.

3.4.2 Non-Functional Requirements

The major non-functional requirements are:

Security

Performance

Usability

Responsiveness

Reliability

Scalability

Maintainability

Compatibility

Data Integrity

Availability

3.4.2.1 Security

The system shall protect:

User accounts

Passwords

Emergency incidents

Location information

Emergency contacts

Administrative information

Risk-related information

JWT-based authorization and secure password hashing shall be used for protected operations.

3.4.2.2 Performance

Common operations should provide reasonable response time under normal project usage, including:

Login

SOS submission

Resource search

Dashboard loading

Status update

Notification creation

3.4.2.3 Usability

The application should be simple and easy to understand.

Important functions should have clear labels such as:

SOS / Emergency

Submit Request

View Status

Find Hospital

Find Shelter

Emergency Contacts

Admin Dashboard

3.4.2.4 Responsiveness

The application should support:

Desktop

Laptop

Tablet

Mobile phone

The interface should adjust according to screen size.

3.4.2.5 Reliability

After successful submission, the incident should remain stored in the database.

The system should handle failed external services without corrupting the incident record.

3.4.2.6 Scalability

The system should allow additional users, incidents, resources and administrative records without requiring a complete redesign.

3.4.2.7 Maintainability

The application should be divided into logical modules such as:

Authentication

User Management

SOS/Incident Management

Location

Facilities

Rescue Teams

Notifications

Risk Assessment

Administration

Analytics

3.4.2.8 Compatibility

The web application should work with commonly used modern browsers.

3.4.2.9 Data Integrity

The system should maintain accurate relationships between database records.

For example, an incident should reference a valid user and an available location record where applicable.

3.4.2.10 Availability

The application should remain available when the hosting environment, database, network and required external services are operational.

3.4.3 User Requirements

Raksha mainly supports two primary system roles:

1. User

The user is the person who uses Raksha to create emergency incidents and access available safety information.

Users shall be able to:

Register

Login

Logout

Manage profile

Manage emergency contacts

Create SOS incidents

Select incident type

Enter incident description

Share location where available

View incident status

View incident history

View notifications

Search emergency resources

Submit unsafe-area reports

2. Administrator

The administrator manages system information and monitors emergency incidents.

Administrators shall be able to:

Login securely

View incidents

View incident details

View available locations

Update incident status

Manage facilities

Manage rescue teams

Manage assignments

Review unsafe-area reports

Manage risk-zone information

View dashboard statistics

Generate reports

Monitor administrative activities

3.4.4 User vs Administrator

Function	User	Administrator
Register	Yes	Usually pre-created
Login	Yes	Yes
Manage Profile	Yes	Limited
Create SOS	Yes	Optional
View Own Incidents	Yes	Yes
View All Incidents	No	Yes

Update Incident Status	No	Yes
Manage Facilities	No	Yes
Manage Rescue Teams	No	Yes
Create Assignments	No	Yes
View Notifications	Yes	Yes
Submit Unsafe Area Report	Yes	Yes
Verify Reports	No	Yes
Manage Risk Zones	No	Yes
View Dashboard	Limited	Yes
Analytics	Limited	Yes
Generate Reports	No/Limited	Yes

3.4.5 System Constraints

The following constraints apply to Raksha:

The system requires internet connectivity for normal web-based functionality.

Location functionality depends on browser/device permission.

Location accuracy depends on the device and available location services.

Map functionality depends on the configured map service.

External email, SMS and WhatsApp services depend on provider availability and configuration.

Database availability affects application functionality.

Hosting performance affects response time.

Unauthorized users must not access protected information.

Location information must be protected.

Emergency response cannot be guaranteed by the software.

Automatic police FIR generation is not assumed.

Exact rescue ETA is not assumed.

24×7 police or ambulance integration is not assumed unless actually implemented.

Real-time hospital bed availability is not assumed unless connected to a reliable source.

Offline emergency functionality is not assumed unless specifically implemented.

Risk assessment results are software-generated decision-support information and are not guaranteed predictions.

Notification delivery depends on configured external communication providers.

3.4.6 Hardware Requirements

Raksha does not require specialized hardware.

Component/hardware	Requirement
Processor	Intel Core i5 / AMD Ryzen 5 or equivalent

RAM	Minimum 8 GB
Storage	Minimum 256 GB SSD
Display	1080p or higher recommended
Network	Stable Internet connection
Development Device	Normal laptop or desktop
Mobile Device	Smartphone with location capability recommended

3.4.7 Software Requirements

The final software stack should reflect the technologies actually used in the implemented project.

Component	Technology
Frontend	React.js
Frontend Language	TypeScript
Build Tool	Vite
Styling	Tailwind CSS
Backend Runtime	Node.js
Backend Framework	Express.js
API Architecture	REST API
AI Backend	Python
AI Framework	FastAPI
AI Integration	Configured AI Service API
Database	MongoDB/superbase
ODM	Mongoose
Authentication	JSON Web Token (JWT)
Password Security	bcrypt
Location	Browser/Device Geolocation API
Maps	Configured Map Service
Communication	Configured Email/SMS/WhatsApp Providers
Development Environment	Visual Studio Code, Bolt
Version Control	Git, GitHub
Testing	Jest / Supertest, where implemented
Deployment	Configured Cloud Hosting Environment

3.4.8 Database Requirement and Collection Mapping

Raksha uses MongoDB for storing application data. Mongoose can be used at the application layer to define schemas and validate database records.

MongoDB does not use traditional relational foreign keys in the same way as a relational database. Therefore, relationships between records are maintained through application-level references and validation.

The proposed database contains the following collections:

No.	Collection	Purpose
1	Users	User accounts and profile

		information
2	EmergencyContacts	User emergency contacts
3	Incidents	SOS and emergency incident records
4	Locations	Geographic location information
5	Notifications	Notification records and delivery status
6	RescueTeams	Registered response-team information
7	RescueAssignments	Incident-to-team assignment records
8	IncidentUpdates	Incident history and status updates
9	UnsafeAreaReports	User-submitted unsafe-area reports
10	RiskZones	Structured risk-zone information
11	RiskAssessments	Risk scores and assessment results
12	Facilities	Hospitals, shelters, police/fire stations and relief centres
13	DisasterCategories	Emergency/disaster categories
14	AdminLogs	Administrative activity records
15	Reports	Generated reports and analytics records

3.5.1 Planning and Scheduling

Sr. No.	Project Activity	Start Date	End Date	Duration	Project Done By
1	Project Planning & Requirements	15-Jun-2026	22-Jun-2026	8 Days	Vivek & Vaibhav – Topic selection, planning and requirements
2	System Analysis & Design	23-Jun-2026	28-Jun-2026	6 Days	Vivek – System flow; Vaibhav – UML and design
3	Database Design	29-Jun-2026	03-Jul-2026	5 Days	Vivek – Database structure; Vaibhav – Collections

					and fields
4	UI & Frontend Design	04-Jul-2026	27-Jul-2026	24 Days	Vivek – User interface; Vaibhav – Admin interface
5	Backend Development	09-Jul-2026	25-Jul-2026	17 Days	Vivek – APIs; Vaibhav – Backend and database connection
6	SOS & Location Module	28-Jul-2026	03-Aug-2026	7 Days	Vivek – SOS; Vaibhav – Location features
7	Emergency & Rescue Modules	04-Aug-2026	14-Aug-2026	11 Days	Vivek – Resource search; Vaibhav – Rescue team and assignments
8	Risk & Disaster Management	15-Aug-2026	21-Aug-2026	7 Days	Vivek & Vaibhav – Risk zones, assessment and disaster features
9	Admin Dashboard & Integration	22-Aug-2026	03-Sep-2026	13 Days	Vivek – System support; Vaibhav – Dashboard; Both – Integration
10	Testing & Debugging	04-Sep-2026	11-Sep-2026	8 Days	Vivek & Vaibhav – Testing and error fixing
11	Documentation & Final Review	12-Sep-2026	27-Sep-2026	16 Days	Vivek – Requirements; Vaibhav – Design; Both – Final review
12	Final Corrections, Deployment & Submission	28-Sep-2026	30-Sep-2026	3 Days	Vivek & Vaibhav – Final corrections, deployment and

					submission
--	--	--	--	--	------------

3.5.1 Gantt Chart

The Gantt Chart shows the planned schedule of the Raksha project, including requirement analysis, system design, development, database implementation, testing, documentation, and final deployment. It helps monitor the progress and completion of each project activity.

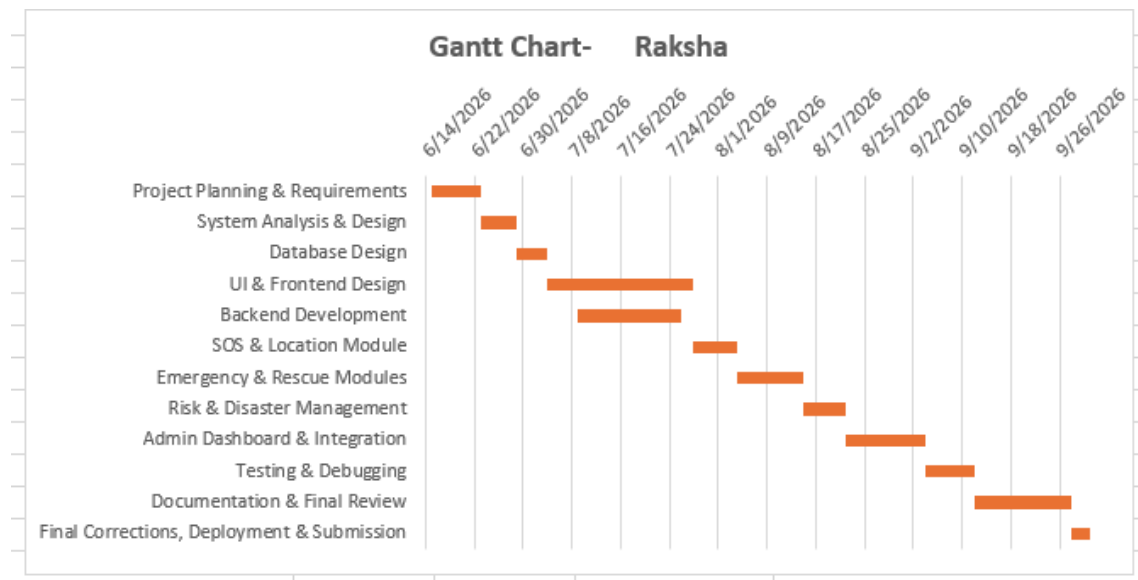


Figure 3.1: Gantt chart - planned project schedule (15 Jun to 30 Sep 2026).

3.5.2 PERT CHART

PERT (Program Evaluation and Review Technique) is used to estimate the time required to complete the Raksha – Women Safety and Disaster Management System. It represents the major project activities in a network form and shows the sequence in which the activities are carried out.

Optimistic Time (O): Minimum expected time

Most Likely Time (M): Normal expected time

Pessimistic Time (P): Maximum expected time

Expected Time is calculated as:

$$TE = (O + 4M + P) / 6$$

Standard Deviation:

$$S.D. = (P - O) / 6$$

PERT Time Estimation Table – RAKSHA

Task	Project Activity	Optimistic Time (O)	Most Likely Time (M)	Pessimistic Time (P)	Expected Time (TE)	S.D.
T1	Project Planning & Requirements	6	8	10	8.00	0.67

T2	System Analysis & Design	5	6	8	6.17	0.50
T3	Database Design	4	5	6	5.00	0.33
T4	UI & Frontend Design	20	24	28	24.00	1.33
T5	Backend Development	14	17	20	17.00	1.00
T6	SOS & Location Module	6	7	9	7.17	0.50
T7	Emergency & Rescue Modules	9	11	13	11.00	0.67
T8	Risk & Disaster Management	6	7	9	7.17	0.50
T9	Admin Dashboard & Integration	11	13	15	13.00	0.67
T10	Testing & Debugging	6	8	10	8.00	0.67
T11	Documentation & Final Review	14	16	18	16.00	0.67
T12	Final Corrections, Deployment & Submission	2	3	4	3.00	0.33

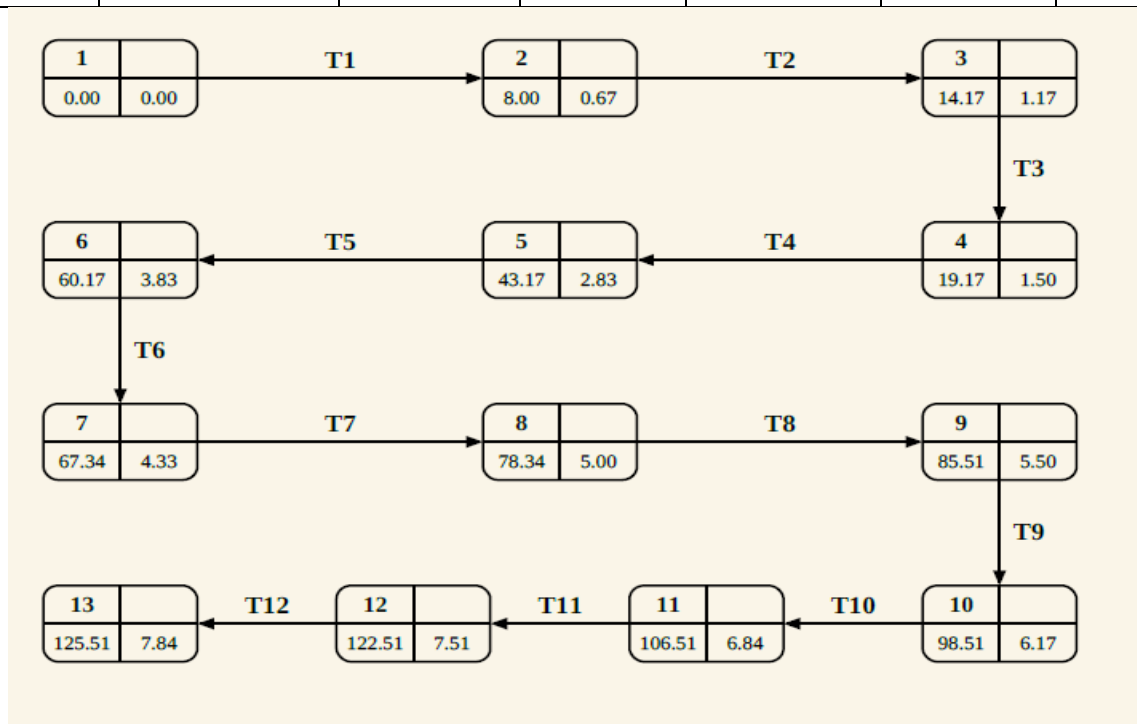


Figure 3.2: PERT network chart - RAKSHA project, with the expected times (T1-T12).

3.6 Conceptual Models

The conceptual model of Raksha Safe shows how the main parts of the system work together. A user can register, add emergency contacts, report an emergency, and use the SOS feature. The system stores the incident and available location details, and shows nearby rescue resources such as police, hospitals, fire services, and volunteers. The AI module provides a basic risk assessment to support the emergency process. The admin manages incidents, users, rescue teams, assignments, notifications, and reports. Overall, the model shows how information moves between the user, system, AI module, rescue resources, and admin.

3.6.1 ER Diagram

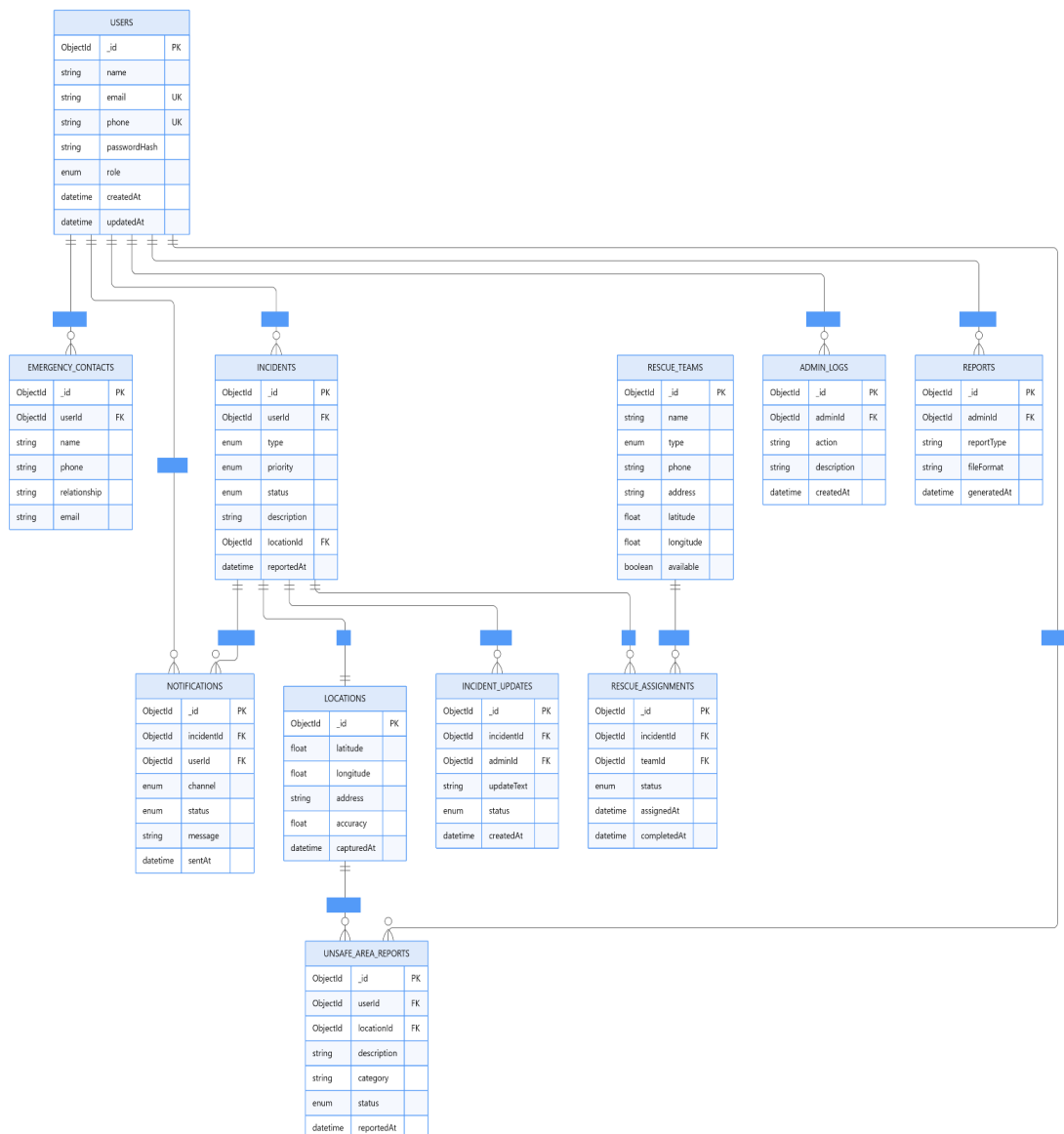


Figure 3.3: ER diagram - conceptual model of RakshaSafe.

3.6.2 Data Dictionary

The Data Dictionary describes the fields, data types and constraints of the major MongoDB collections used in the Raksha – Women Safety and Disaster Management System. The

system contains 15 main collections for managing users, emergency incidents, locations, notifications, rescue resources, risk information, administrative activities and reports.

1. Users

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	Name	String	Required
3	Email	String	Required, Unique
4	phone	String	Required, Unique
5	passwordHash	String	Required, Hashed
6	Role	String	Required, Enum: USER / ADMIN
7	language	String	Optional
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

2. Emergency Contacts

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	userId	ObjectId	Reference → Users
3	Name	String	Required
4	Phone	String	Required
5	Email	String	Optional
6	relationship	String	Optional
7	notifyViaSms	Boolean	Required
8	notifyViaEmail	Boolean	Required
9	createdAt	Date	Auto-generated
10	updatedAt	Date	Auto-generated

3. Incidents

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	userId	ObjectId	Reference → Users
3	Type	String	Required, Safety / Disaster
4	category	String	Required
5	description	String	Required
6	priority	String	Required, LOW / MEDIUM / HIGH / CRITICAL
7	status	String	Required, Enum
8	locationId	ObjectId	Reference → Locations, Optional
9	createdAt	Date	Auto-generated

10	resolvedAt	Date	Optional
----	------------	------	----------

4. Locations.

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	latitude	Number	Required
3	longitude	Number	Required
4	address	String	Optional
5	City	String	Optional
6	state	String	Optional
7	country	String	Optional
8	accuracy	Number	Optional
9	createdAt	Date	Auto-generated
10	updatedAt	Date	Auto-generated

5. Notifications

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	incidentId	ObjectId	Reference → Incidents
3	contactId	ObjectId	Reference → EmergencyContacts, Optional
4	channel	String	Required, Email / SMS / WhatsApp / In-App
5	Status	String	Required, Enum
6	providerResponse	String	Optional
7	attemptCount	Number	Required
8	lastAttemptAt	Date	Optional
9	createdAt	Date	Auto-generated
10	updatedAt	Date	Auto-generated

6. RescueTeams

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	name	String	Required
3	teamType	String	Required, Police / Medical / Fire / NGO / Volunteer
4	phone	String	Required
5	email	String	Optional
6	isActive	Boolean	Required
7	specializations	Array	Optional
8	createdAt	Date	Auto-generated

9	updatedAt	Date	Auto-generated
---	-----------	------	----------------

7. RescueAssignments

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	incidentId	ObjectId	Reference → Incidents
3	teamId	ObjectId	Reference → RescueTeams
4	assignedBy	ObjectId	Reference → Users
5	assignedAt	Date	Auto-generated
6	status	String	Required, Enum
7	notes	String	Optional
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

8. IncidentUpdates

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	incidentId	ObjectId	Reference → Incidents
3	updatedBy	ObjectId	Reference → Users
4	statusFrom	String	Optional
5	statusTo	String	Required
6	comment	String	Optional
7	createdAt	Date	Auto-generated

9. UnsafeAreaReports

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	reportedBy	ObjectId	Reference → Users
3	locationId	ObjectId	Reference → Locations
4	category	String	Required
5	description	String	Required
6	severity	String	Required
7	isVerified	Boolean	Required
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

10. RiskZones

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	name	String	Required

3	riskLevel	String	Required
4	geometry	Object	Required
5	factors	Array	Optional
6	lastAssessedAt	Date	Optional
7	isActive	Boolean	Required
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

11. RiskAssessments

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	locationId	ObjectId	Reference → Locations
3	riskScore	Number	Required, Range: 0-100
4	riskLevel	String	Required, LOW / MEDIUM / HIGH / CRITICAL
5	modelVersion	String	Required
6	inputFactors	Array	Optional
7	assessedAt	Date	Auto-generated
8	createdAt	Date	Auto-generated

12. Facilities

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	name	String	Required
3	facilityType	String	Required, Enum
4	locationId	ObjectId	Reference → Locations
5	phone	String	Required
6	capacity	Number	Optional
7	isOperational	Boolean	Required
8	operatingHours	String	Optional
9	createdAt	Date	Auto-generated
10	updatedAt	Date	Auto-generated

13. Disaster Categories

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	Name	String	Required
3	Code	String	Required, Unique
4	description	String	Optional
5	defaultPriority	String	Required
6	requiresResponseTeam	Boolean	Required

7	isActive	Boolean	Required
8	createdAt	Date	Auto-generated
9	updatedAt	Date	Auto-generated

14. AdminLogs

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	adminId	ObjectId	Reference → Users
3	action	String	Required
4	targetType	String	Required
5	targetId	ObjectId	Optional
6	details	String	Optional
7	ipAddress	String	Optional
8	createdAt	Date	Auto-generated

15. Reports

Sr. No.	Field	Type	Constraint
1	_id	ObjectId	Primary Key, Auto-generated
2	generatedBy	ObjectId	Reference → Users
3	Title	String	Required
4	reportType	String	Required
5	filters	Object	Optional
6	dataSnapshot	Object	Optional
7	format	String	Required, PDF / CSV / JSON
8	createdAt	Date	Auto-generated
9	expiresAt	Date	Optional

3.6.3 DFD LEVEL 0 — CONTEXT DIAGRAM

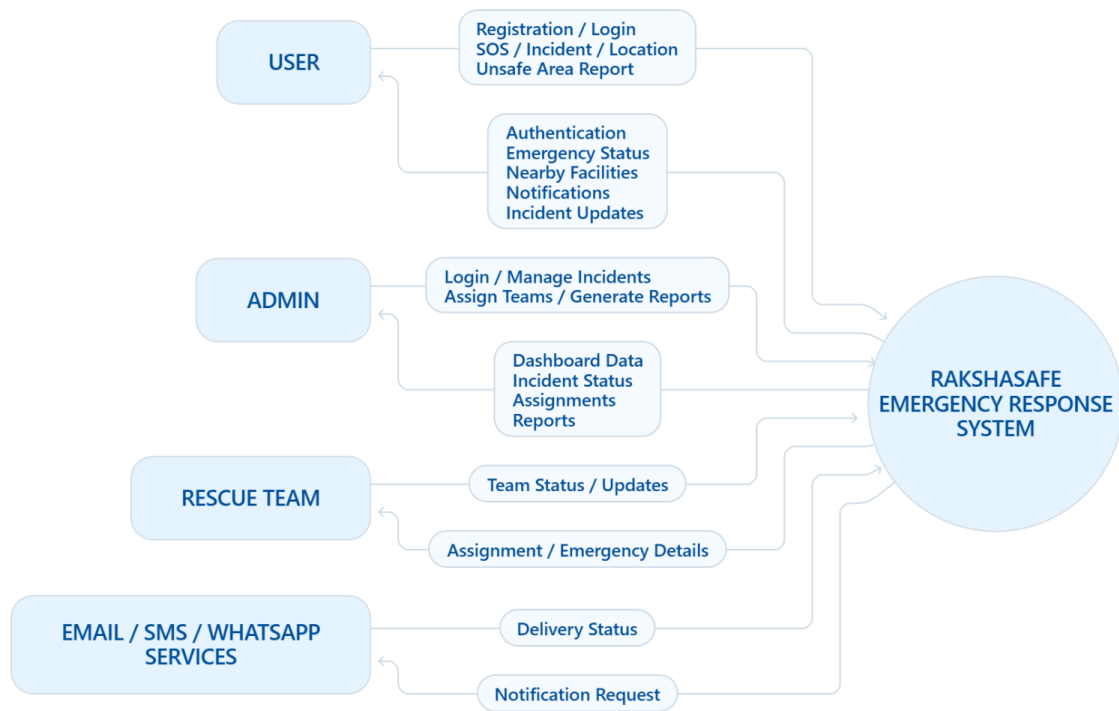


Figure 3.4: DFD Level 0 - context diagram of RakshaSafe.

DFD LEVEL 1 — MAIN MODULES

CHAPTER 4

SYSTEM DESIGN

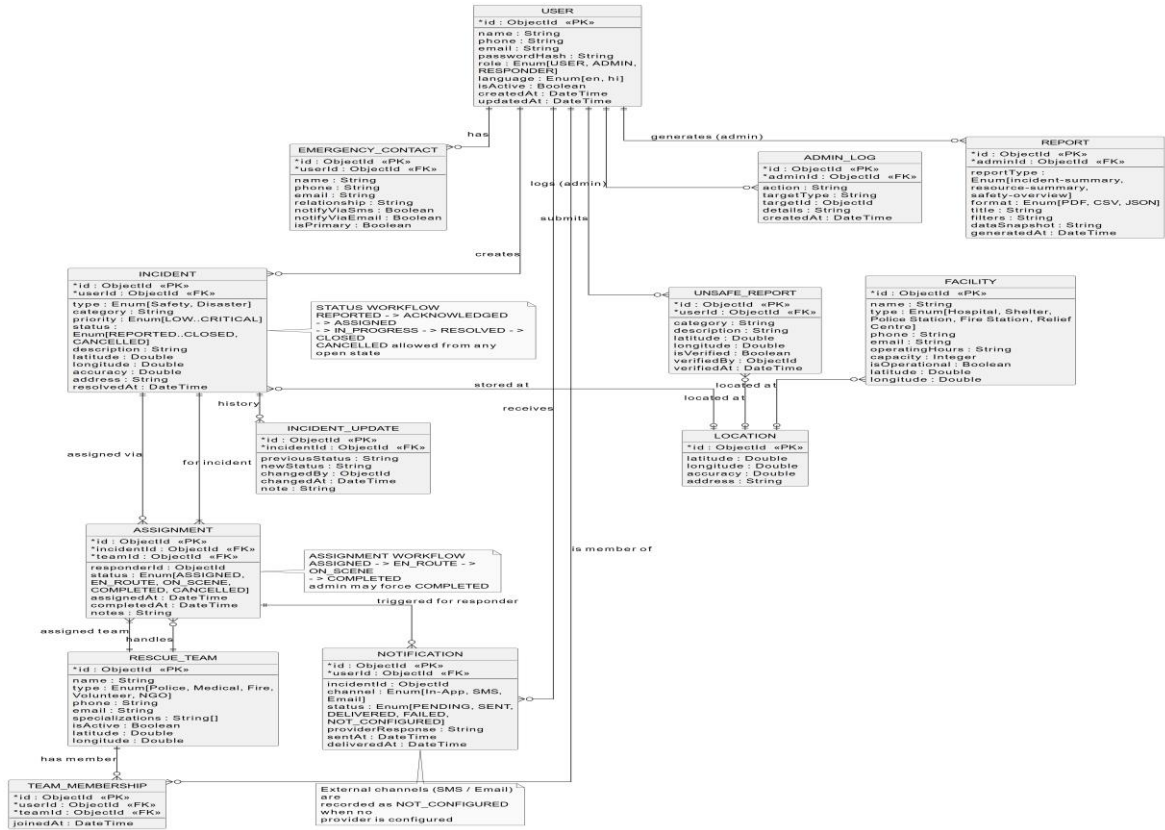


Figure 4.1: Entity-Relationship model (13 entities).

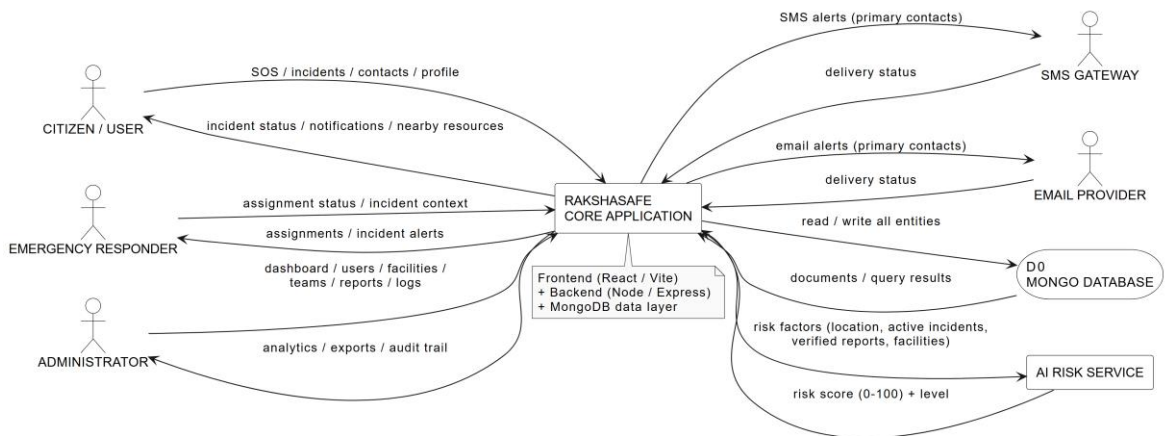


Figure 4.2: DFD Level 0 - context diagram.

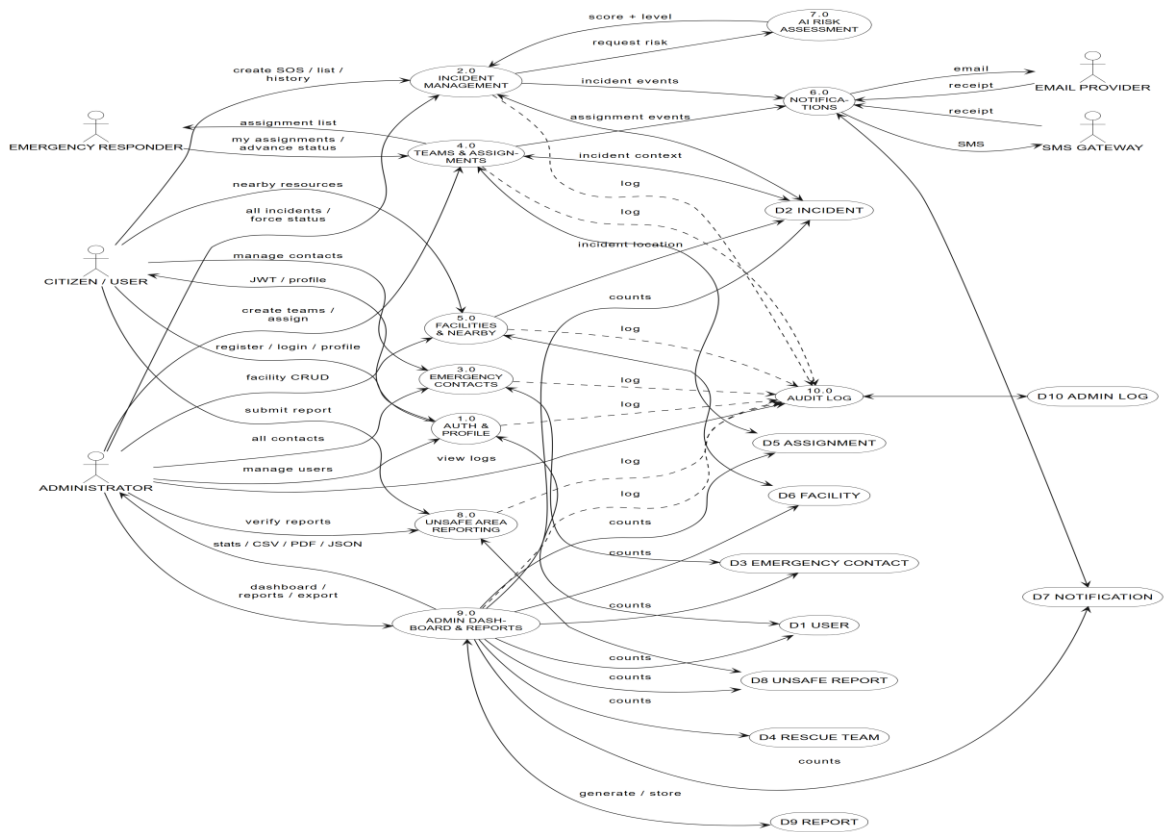


Figure 4.3: DFD Level 1 - major processes.

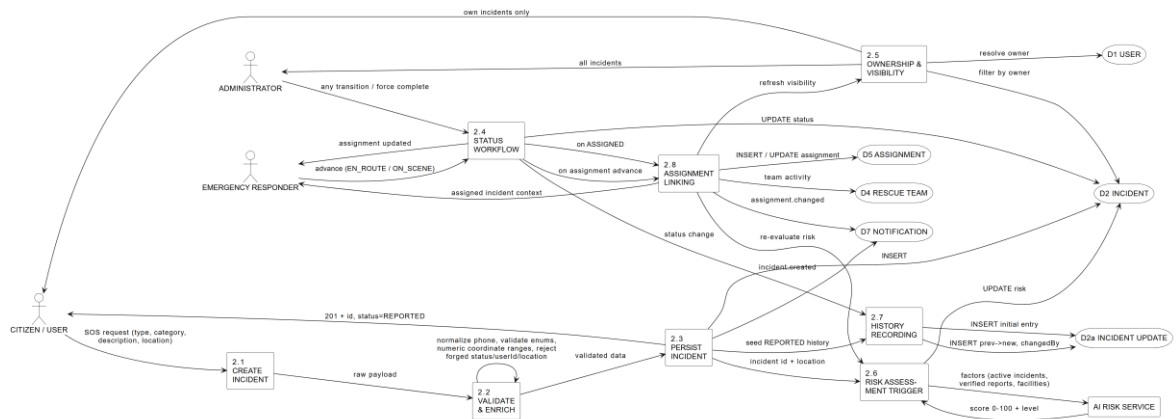


Figure 4.4: DFD Level 2 - incident management.

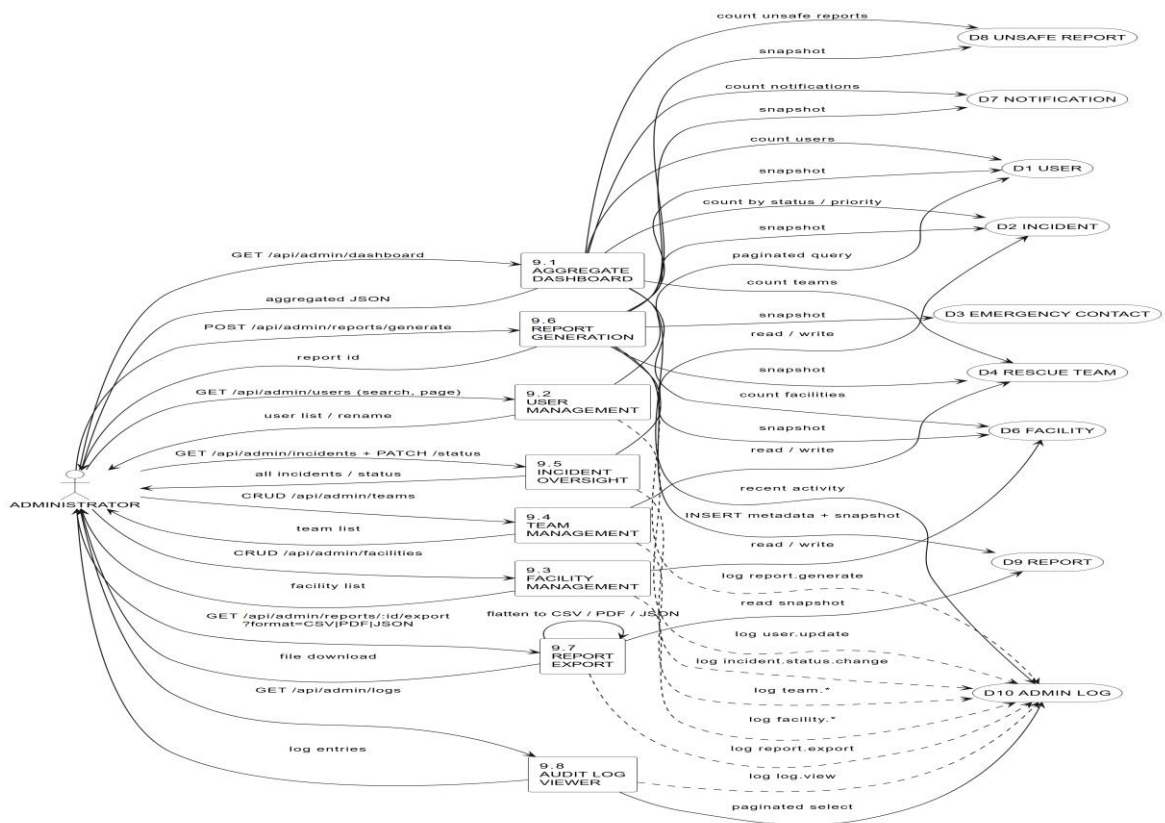


Figure 4.5: DFD Level 2 - admin dashboard and reports.

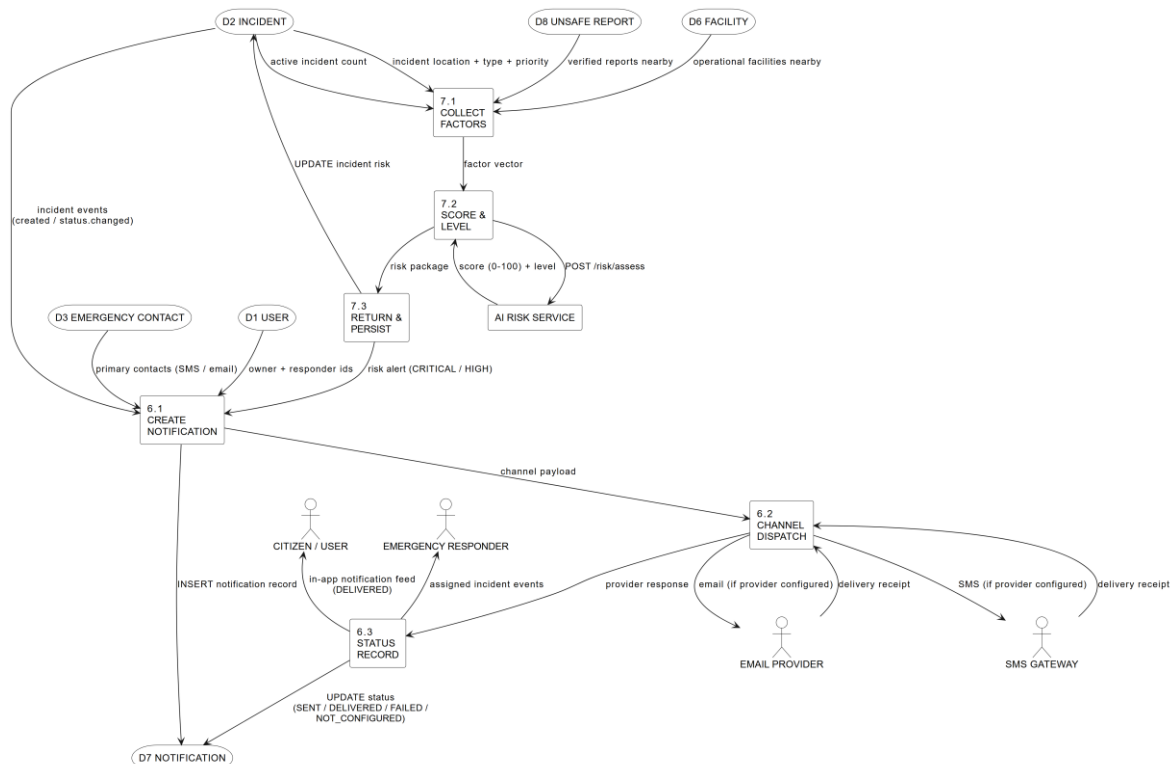


Figure 4.6: DFD Level 2 - notifications and AI risk.

Introduction

Before looking at any single screen or service, it helps to see the whole system at once. RakshaSafe is not one big program but a set of focused modules that each own a small, well-defined job: signing users in, recording emergencies, attaching real locations, keeping

emergency contacts, organising facilities and rescue teams, sending notifications, scoring risk and letting administrators supervise everything. This chapter walks through that design the way the software is actually built, so every section below traces back to real pages, real API routes and real database collections.

The table below lists every module of the system with what it covers and where it lives in the implementation. It is the map for the rest of the chapter: Section 4.1 explains how the modules work together, Section 4.2 describes each interface in detail, and Section 4.3 covers deployment and security.

Module	What it covers	Main screens / API
Authentication	Registration, login, logout and JWT sessions for all three roles	Login, Register; /api/auth/*
User Management	Profiles, language preference and account state	Profile; /api/auth/me, /api/auth/profile
SOS / Incident Management	Emergency reporting, status workflow, history and ownership	SOS, Incident Detail, Dashboard; /api/incidents/*
Location	Coordinate capture, stored points and readable addresses	SOS location step, Location cards; /api/geocode/*
Emergency Contacts	Per-user contacts with call links and alert channels	Emergency Contacts, Admin Emergency Contacts; /api/emergency-contacts/*, /api/admin/emergency-contacts
Emergency Resources	Searchable directory and nearby lookup of help	Resources, Nearby sections; /api/facilities, /api/nearby-resources, /api/nearby
Facilities	Hospitals, shelters, police and fire stations, relief centres	Resources, Admin Facilities; /api/facilities, /api/admin/facilities
Rescue Teams	Registered teams with type, contact, members and base	Resources, Admin Teams; /api/rescue-teams, /api/admin/rescue-teams
Rescue Assignments	Incident-to-team assignment records and their lifecycle	Incident Detail, Admin incident detail; /api/admin/incidents/:id/assignments, /api/responder/assignments
Unsafe-Area Reporting	User reports with admin verification queue	Unsafe Reports, Admin Unsafe Reports; /api/unsafe-reports/*, /api/admin/unsafe-reports
Risk Zones	Administrator-maintained boundary records with risk levels	Admin masters; RiskZones collection
Risk Assessment (AI)	Deterministic raksha-risk-v1 scoring of incident locations	Risk panels; /api/incidents/:id/risk, FastAPI POST /risk/assess

Notifications	In-app and provider-channel delivery records with status	Notifications; /api/notifications, /api/responder/notifications
Weather Enrichment	Informational current weather for incident coordinates	Weather cards; /api/weather (Open-Meteo, no key)
Administration	Dashboard, users, reports, activity logs and analytics	Admin Dashboard, Users, Reports; /api/admin/*

4.1 System Architecture

The system design of RakshaSafe – Women Safety and Disaster Emergency Response System describes how the different parts of the system work together to record emergency incidents, associate them with real locations, organise emergency resources, support rescue assignments, produce assistive risk assessments, deliver notifications and provide administration. The design keeps three concerns separate: the React web application that users and administrators operate, the Node.js backend that enforces authentication, validation and record relationships, and the Python AI service that scores risk. MongoDB stores every persistent record, and the backend is the only writer of application data.

4.1.1 Overall System Working

The overall working of RakshaSafe is divided into the following stages:

1. User Authentication

a. A new user registers with name, email, phone and password. An existing user, administrator or responder signs in with the registered email or phone number together with the password. b. Passwords are never stored in plain text; they are hashed with bcrypt and verified on login. c. On successful authentication the backend issues a JSON Web Token (JWT). The frontend stores the session and redirects by role: USER accounts reach the user dashboard, ADMIN accounts reach the admin dashboard, and RESPONDER accounts reach the responder workspace.

2. SOS and Incident Creation

a. The signed-in user opens the SOS page and describes the emergency: incident type (Safety or Disaster), a category such as Women Safety, Medical Emergency, Accident, Fire, Flood, Earthquake or Other Emergency, a free-text description and a priority of LOW, MEDIUM, HIGH or CRITICAL. b. The backend validates the request, creates an Incidents record with status REPORTED and returns the stored incident. The incident immediately appears in the user dashboard and in the admin incident list.

3. Location Capture and Association

a. Where the user grants permission, the browser Geolocation API supplies latitude, longitude and accuracy. The coordinates are sent with the incident request and stored as a Locations record linked through locationId. b. A server-side reverse-geocode step resolves a readable address, city, district, state, postal code and country for display. If permission is denied or the position is unavailable, the incident is still recorded without coordinates; no false location is ever assumed or stored.

4. Emergency Resource Lookup

a. Around a real query point the system lists nearby Facilities (Hospital, Shelter, Police Station, Fire Station, Relief Centre) and active RescueTeams, nearest first with straight-line distances. b. Each card shows the values actually present in the source record: name, type, address or “Address unavailable”, and distance. A Call button using a tel: link is rendered only when a real phone number exists, and a Directions link is built only from real coordinates. Where an external places provider is configured, its live results are merged with stored records; when it is unavailable the page honestly reports that and continues with stored resources.

5. Rescue Assignment

a. An administrator assigns a registered, active rescue team to an incident, optionally with notes. b. Assignment status follows ASSIGNED → EN_ROUTE → ON_SCENE → COMPLETED, with CANCELLED where applicable. Every change is a software record in RescueAssignments; the system never claims automatic dispatch, guaranteed response or an exact arrival time.

6. AI Risk Assessment

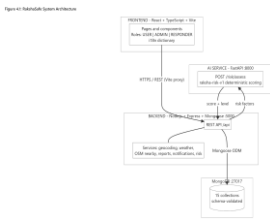
a. For an incident that has stored coordinates, the user (or the admin view of the same incident) can request an assistive risk assessment. The backend assembles fixed factors — priority, incident type, counts of verified and unverified unsafe-area reports nearby, and other active incidents nearby — and posts them to the FastAPI AI service. b. The deterministic raksha-risk-v1 model returns a risk score from 0 to 100, a risk level of LOW, MEDIUM, HIGH or CRITICAL, the model version, the input factors and the assessment time. The backend validates this response and stores it in RiskAssessments, then serves the stored record to both user and admin views. The result is decision support only and is always shown with a disclaimer.

7. Notifications

a. Incident lifecycle events — creation, acknowledgement, assignment, status changes, resolution and cancellation — create Notification records for the incident owner and, where configured, for emergency-contact channels. b. Each notification carries its channel (Email, SMS, WhatsApp or In-App), a status of QUEUED, SENT, DELIVERED, FAILED, NOT_CONFIGURED or UNAVAILABLE, the provider response, attempt counts and timing. Delivery depends on the configured providers; a delivery failure never corrupts the incident record.

8. Administration and Reporting

a. Administrators work through the admin dashboard: users, emergency contacts of any user, incidents with reporter, location, history and assignments, facilities, rescue teams and their members, unsafe-area report verification, risk-zone records, generated reports in PDF, CSV or JSON, and the admin activity log. b. Incident status follows REPORTED → ACKNOWLEDGED → ASSIGNED → IN_PROGRESS → RESOLVED → CLOSED, with CANCELLED where applicable, and each transition is stored as an IncidentUpdates record with an optional comment.



RakshaSafe System Architecture

4.1.2 Technology Used

The technologies below are the ones actually used in the implementation. Each choice matches a layer of the system, and no component in the table is aspirational: the frontend is the React application in the repository, the backend is the Express API, the database is MongoDB accessed through Mongoose, and the AI service is the FastAPI application in the ai-service folder.

Component	Technology	Purpose
Frontend	React.js, TypeScript, Vite, Tailwind CSS	User and admin web interface
Backend	Node.js, Express.js, TypeScript	REST APIs and business logic
Authentication	JWT, bcrypt	Session tokens and password security
Database	MongoDB, Mongoose ODM	Persistent records and validation
AI Service	Python, FastAPI	Deterministic risk scoring

AI Model	raksha-risk-v1 (rule-based scoring)	Risk score and level, decision support only
Location	Browser Geolocation API	Latitude, longitude and accuracy capture
Maps	OpenStreetMap, Leaflet	Incident and resource map display
Communication	Configured Email / SMS / WhatsApp providers	Notification delivery where configured
Weather	Open-Meteo (no key)	Informational weather enrichment

4.1.3 Database Design

MongoDB holds fifteen collections. Relationships are ObjectId references maintained by Mongoose schemas rather than relational foreign keys, and critical integrity is enforced through schema validation. The table below names each collection with its purpose and principal references; the detailed field lists follow Chapter 3 and the model files, so only the important references are repeated here.

Collection	Purpose	Principal references
Users	Accounts and profiles	Unique email and phone
EmergencyContacts	User emergency contacts	userId → Users
Incidents	SOS and emergency records	userId → Users; locationId → Locations (optional)
Locations	Geographic points and resolved address	Standalone; referenced by incidents, facilities, reports
Notifications	Delivery records and status	incidentId → Incidents; contactId → EmergencyContacts (optional)
RescueTeams	Registered response teams and members	members → Users; locationId → Locations (optional)
RescueAssignments	Incident-to-team assignments	incidentId → Incidents; teamId → RescueTeams; assignedBy → Users
IncidentUpdates	Status history and comments	incidentId → Incidents; updatedBy → Users
UnsafeAreaReports	User-submitted unsafe-area reports	reportedBy → Users; locationId → Locations
RiskZones	Structured risk-zone records	Standalone boundary records managed by admins
RiskAssessments	Risk scores and assessment results	locationId → Locations
Facilities	Hospitals, shelters, stations, relief centres	locationId → Locations
DisasterCategories	Maintained incident categories	Referenced by category name/code
AdminLogs	Administrative activity	adminId → Users; targetId

	records	(various)
Reports	Generated analytics records	generatedBy → Users

The relationships that matter for the design are short: a user owns incidents, contacts and unsafe-area reports; an incident optionally points at one location and accumulates updates, assignments, notifications and — through its location — risk assessments; facilities and rescue teams each point at locations; and every administrative action appends to AdminLogs. Nothing in the design requires joins across databases, because there is only one database.

4.1.4 API Design

The backend exposes a REST API under /api with a uniform JSON envelope: success responses carry a data object, while failures carry a message and, for validation problems, a field-level issues list. Every router declares its own guard chain, so the access rule of an endpoint can be read directly from the route files: public authentication endpoints stand alone, user endpoints combine requireAuth with requireDb and owner scoping, responder endpoints add requireResponder, and the whole /api/admin tree combines requireAuth, requireAdmin and requireDb. Creation endpoints for incidents and unsafe-area reports additionally carry per-IP rate limits against flooding. The table below lists the implemented endpoints; enrichment lookups for live maps, weather and geocoding deliberately omit requireDb so informational features stay available while MongoDB is unreachable.

Method and path	Access	Purpose
POST /api/auth/register	Public, database	Create a USER account
POST /api/auth/login	Public, database, rate-limited	Authenticate and issue a JWT
POST /api/auth/logout	Public	End the client session
GET /api/auth/me	Authenticated, database	Return the current account
PATCH /api/auth/profile	Authenticated, database	Update the own profile
POST /api/emergency-contacts	Authenticated, database, owner	Add an own emergency contact
GET /api/emergency-contacts	Authenticated, database, owner	List the own emergency contacts
PATCH /api/emergency-contacts/:id	Authenticated, database, owner	Update an own contact
DELETE /api/emergency-contacts/:id	Authenticated, database, owner	Delete an own contact
POST /api/incidents	Authenticated, database, rate-limited	Create an SOS incident
GET /api/incidents	Authenticated, database, owner	List the own incidents
GET /api/incidents/:id	Authenticated, database, owner	Incident detail with location

PATCH /api/incidents/:id	Authenticated, database, owner	Edit an own early incident
DELETE /api/incidents/:id	Authenticated, database, owner	Delete an own early incident
GET /api/incidents/:id/updates	Authenticated, database, owner	Status history of the incident
POST /api/incidents/:id/risk	Authenticated, database, owner	Request a risk assessment
GET /api/incidents/:id/risk	Authenticated, database, owner	Read stored assessments

... remaining 48 rows can be found in the chapter as provided.

Two properties of this table deserve emphasis for the viva. First, ownership is always derived from the JWT identity on the server; incident and contact identifiers in the URL select records, they never confer access, and a user who guesses another account's identifier receives a not-found response rather than the record. Second, failures are compartmentalised: an unreachable AI service, map provider or weather upstream produces a retryable error for that panel alone, while incident creation, status management and stored-resource lookup continue to work.

4.1.5 Risk Scoring Design

Risk scoring is the only component called “AI” in this project, and the name needs care: the service performs no machine learning, interprets no text and calls no external model. The Node.js backend counts bounded facts from stored records around the incident location — the priority, the incident type, nearby verified and unverified unsafe-area reports, and other nearby active incidents — and the FastAPI service applies fixed arithmetic documented as model raksha-risk-v1. Because free-text descriptions never cross into the scorer, prompt-injection style input cannot influence the result; only the pre-computed numbers matter.

Factor	Contribution	Cap
Priority base	LOW 15, MEDIUM 35, HIGH 60, CRITICAL 80	Always exactly one base value
Incident-type modifier	Safety +0, Disaster +5	Fixed per type
Verified unsafe reports nearby	+6 each	+30
Unverified reports nearby	+2 each	+10
Other active incidents nearby	+3 each	+15
Total score	Sum of the above, clamped to 0–100	Integer only

Level bands follow the score directly: below 25 is LOW, below 50 is MEDIUM, below 75 is HIGH, and anything at or above 75 is CRITICAL. The response carries the score, the level,

the model version, the five weighted input factors and the assessment timestamp, and the backend re-validates every field — integer score within range, level within the four enum values, non-empty version, factor array and parseable time — before anything is stored in RiskAssessments. An invalid, unreadable or late response becomes a retryable panel error, and the frontend displays the stored record with the decision-support disclaimer on every view.

4.2 System Interface Design

The RakshaSafe interface gives users and administrators a consistent workspace: a navigation bar with the золотые modules for the signed-in role, a content area of cards, tables with search and pagination, explicit loading skeletons, empty states and error states with retry, and small badges for status, priority and levels. Forms use labelled inputs with inline field errors, destructive actions always ask for confirmation, and phone numbers become tel: links while coordinates become map links. The layout adapts from mobile to desktop and honours the light and dark themes. The interfaces below are the ones actually implemented in the frontend pages.

4.2.1 Login Interface

The Login Interface is the entry point for every account. It accepts the registered email address or phone number together with the password, authenticates against the backend, and routes the session to the dashboard that matches the account role. The page stays deliberately plain so that signing in remains fast even on weak connections.

Main elements:

Email-or-phone field for the registered identifier.

Password field with a show/hide visibility option.

Sign In button for authentication.

Create Account link for new users.

Session-expired notice when redirected after a timeout.

Validation:

The identifier must be a recognised email address or phone number; blank or malformed input is rejected locally.

The password must be entered; empty submissions are blocked.

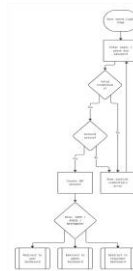
Invalid credentials are rejected by the backend without revealing which half was wrong.

Deactivated accounts are refused with a contact-support message.

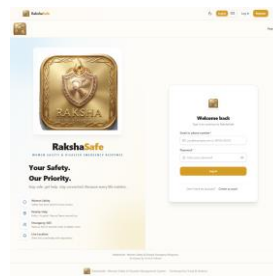
Only an authenticated session can leave this page for the application.

Steps:

1. Enter the registered email address or phone number.
2. Enter the password.
3. Click Sign In.
4. The system validates the credentials and verifies the account is active.
5. On success the user is redirected by role: USER to the user dashboard, ADMIN to the admin dashboard, RESPONDER to the responder workspace.



Login



Login Interface

4.2.2 User Registration Interface

The Registration Interface creates a standard USER account. It collects the minimum profile needed for emergency use — a name, a reachable email and phone number, and a password — and leaves administrative roles to be granted outside this form. Successful registration signs the session in directly, without any email-code step.

Main elements:

Full-name field.

Email address field.

Phone number field.

Password field with strength guidance.

Preferred-language option.

Create Account button and a Back to Sign In link.

Validation:

Name, email, phone and password are all required.

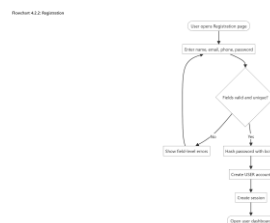
Email must be valid in format and unique; phone must be a valid number and unique.

The password must satisfy the minimum-length and complexity rule.

Duplicate email or phone registrations are rejected with a field-level message.

Steps:

1. Enter the full name.
2. Enter the email address.
3. Enter the phone number.
4. Create a password.
5. Optionally choose the preferred language.
6. Click Create Account.
7. The system validates, stores the hashed password, creates the session and opens the user dashboard.



Registration

The screenshot shows a web form titled "Create your account" on a light orange background. The form includes input fields for "Full name", "Email", "Phone", "Password", and "Confirm password". There is a dropdown menu for "Preferred language" and a "Create account" button. A note at the bottom states "We'll send you emails about our products and services."

Registration Interface

4.2.3 User Dashboard Interface

The Dashboard is the home screen for a signed-in user. Rather than analytics, it answers three questions at a glance: what are my recent incidents, what state is each one in, and where do I go next. Quick actions lead to SOS reporting, emergency contacts, resources, unsafe-area reporting and notifications, so the dashboard behaves as a hub instead of a dead end.

Main elements:

Recent-incidents list with category, priority badge and status badge.

Status summary of the user's own active and resolved incidents.

Quick actions: Report SOS, Emergency Contacts, Find Resources, Report Unsafe Area.

Notifications entry with unread indication.

Shortcuts to profile and language settings.

Validation:

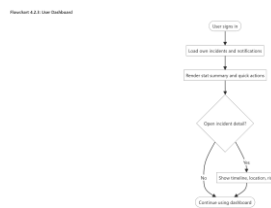
Only the signed-in user's own incidents are listed; no other user's data appears.

Counts are computed from stored records and never include other accounts.

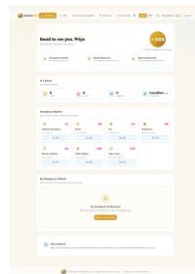
Navigation guards keep signed-out visitors on the login page.

Steps:

1. Sign in as a user.
2. Review the recent-incidents list and their statuses.
3. Use a quick action to start SOS reporting, manage contacts, or browse resources.
4. Open any incident for its full detail, history and risk information.



User Dashboard



Dashboard Interface

4.2.4 SOS and Incident Creation Interface

The SOS Interface turns an emergency into a structured incident record in a few guided steps. It asks for the incident type, category, description and priority first, then offers location capture, and finally shows a review screen before submission. Location is helpful but never mandatory: the incident is created with or without coordinates.

Main elements:

Incident-type selector: Safety or Disaster.

Category selector: Women Safety, Medical Emergency, Accident, Fire, Flood, Earthquake, Other Emergency.

Description area for what is happening and who is affected.

Priority selector: LOW, MEDIUM, HIGH, CRITICAL.

Use My Location button with permission, timeout and accuracy handling.

Review screen showing every entered value before submission.

Validation:

Type, category, description and priority are required; descriptions have a maximum length.

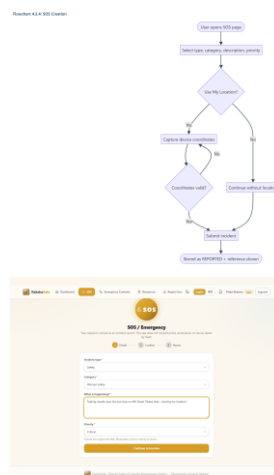
Coordinates, when supplied, must be valid latitude (-90 to 90) and longitude (-180 to 180).

Denied or failed geolocation records the incident without coordinates instead of inventing any.

Double submission is prevented while a request is in flight.

Steps:

1. Open the SOS page and enter type, category, description and priority.
2. Optionally capture the device location or continue without it.
3. Review the entered details on the confirmation screen.
4. Submit the SOS request.
5. The system stores the incident as REPORTED and shows its reference.



SOS Interface

4.2.5 Incident Detail Interface

The Incident Detail Interface is the user's single view of one incident: its facts, its location, nearby help, its risk assessment, any assigned team and its full history. Supplementary sections such as nearby resources, weather and risk load independently, so a failure in one never hides the incident itself.

Main elements:

Header with category, reference id, status badge and priority badge.

Description, reported and resolved timestamps.

Location card with coordinates, accuracy, resolved area and a map link.

Nearby emergency resources with Call and Directions actions.

1. Open an incident from the dashboard.
2. Read the facts, location and history.
3. Use nearby resources or the team contact directly when needed.
4. Request a risk assessment or retry it when the service was unavailable.
5. Edit or delete the incident while it is still manageable.

[illegible]

The Emergency Contacts Interface lets each user keep the people who should be reachable in an emergency. Contacts belong strictly to their owner: the same stored record later appears to administrators with its owner attached, but no user ever sees another user's contacts. A primary flag marks the first contact, and per-contact toggles decide SMS and email notification channels.

Main elements:

Contact list with name, phone, relationship and channel badges.

Primary-contact marker with a one-primary-per-user rule.

Add Contact form: name, phone, email, relationship, SMS and email toggles.

Edit form pre-filled from the stored record.

Delete confirmation naming the contact before removal.

Call buttons using tel: links built from the stored phone number.

Validation:

Name and phone are required; email and relationship are optional.

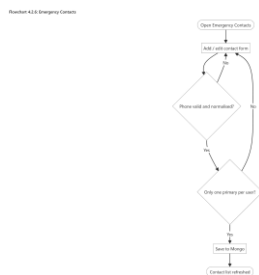
Phone numbers are validated and normalised to the Indian format before saving.

Duplicate-primary violations and invalid numbers return field-level errors.

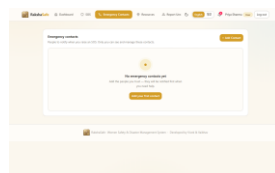
Ownership is taken from the session; a userId supplied by the client is never trusted.

Steps:

1. Open Emergency Contacts.
2. Click Add and enter name, phone and optional details.
3. Save; the contact appears in the list immediately.
4. Edit any contact to correct details or change channel toggles.
5. Delete with confirmation; the row is removed only after the backend confirms.



Emergency Contacts



Emergency Contacts Interface

4.2.7 Emergency Resources Interface

The Resources Interface is a directory of help, not a dispatch console. It searches stored Facilities and active RescueTeams by name, type and operational state, and it can also centre the search on live coordinates to show what is nearest. Every card is honest about what is known and silent about what is not.

Main elements:

Search box with type and operational-state filters.

Resource cards with name, facility or team type, phone and address.

Distance labels computed from real coordinates.

Call buttons only where a genuine phone number is stored.

Directions links only where genuine coordinates are stored.

Source badges distinguishing stored RakshaSafe records from live provider results.

Validation:

Search text is trimmed and length-limited before querying.

Radius values are bounded so external providers cannot be overloaded.

Records without coordinates never receive a fabricated distance.

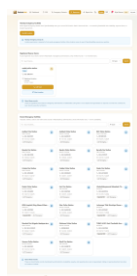
Steps:

1. Open Resources.
2. Enter a search term or pick a facility type.
3. Optionally centre the search on the current location.
4. Call a resource directly or open walking and driving directions.

Flowchart 4.2.7 Resource Lookup



Resource Lookup



Emergency Resources Interface

4.2.8 Unsafe Area Report Interface

The Unsafe Area Report Interface lets users flag places that felt unsafe, with a location, a category, a description and a severity. Reports enter the system unverified: they are visible to administrators for review and they contribute to risk scoring only through the verified and unverified counts the backend computes, never as confirmed danger zones on their own.

Main elements:

Category selector and severity selector.

Description area for what was observed.

Location picker using device coordinates or a searched place.

Own-reports list with verification badges.

Edit and withdraw actions while the report is still unverified.

Validation:

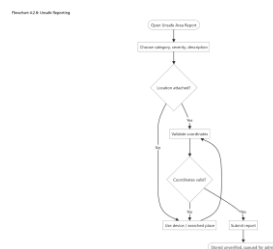
Category, description, severity and a valid location are required.

Coordinates are validated before submission; invalid positions are rejected.

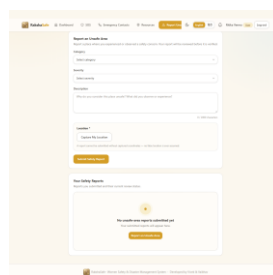
Only the reporting user can edit or withdraw their own report.

Steps:

1. Open the unsafe-area reporting page.
2. Choose the category and severity and describe the concern.
3. Attach the location from the device or search.
4. Submit; the report is stored as unverified and queued for admin review.



Unsafe Reporting



Unsafe Area Report Interface

4.2.9 Risk Assessment Interface

The Risk Assessment Interface presents the stored AI assessment for an incident's location. It shows the backend-computed score out of 100, the LOW, MEDIUM, HIGH or CRITICAL level, the raksha-risk-v1 model version, the weighted input factors and the assessment time, together with the history of earlier assessments. The panel never recalculates anything itself: the backend score is the single source of truth.

Main elements:

Risk score panel with score, level badge, model version and assessment time.

Weighted factor list: priority base, type modifier, verified reports, unverified reports, active incidents.

Assess Risk button for incidents with stored coordinates.

Try Again button when the AI service was unreachable or timed out.

Previous-assessments list, newest first.

Persistent decision-support disclaimer on every result.

Validation:

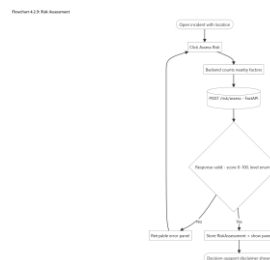
Assessment requires stored coordinates; incidents without a location explain that instead of scoring.

Backend responses are strictly validated: scores must be integers 0–100 and levels must be one of the four enum values.

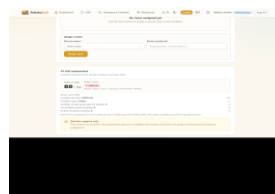
Invalid or unreadable AI responses surface as a retryable error, never as a fabricated score.

Steps:

1. Open an incident that has a location.
2. Click Assess Risk.
3. Read the score, level, factors and time once stored.
4. Revisit the history on refresh; saved assessments persist.
5. Use Try Again if the service was temporarily unavailable.



Risk Assessment



Risk Assessment Interface

4.2.10 Notifications Interface

The Notifications Interface keeps the user informed about their own incidents without demanding attention. Entries record what happened, through which channel, and whether delivery succeeded, so the user can distinguish a recorded event from a delivered message.

Main elements:

Notification list grouped by incident with channel badges.

Status badges: QUEUED, SENT, DELIVERED, FAILED, NOT_CONFIGURED, UNAVAILABLE.

Unread markers with a mark-as-read behaviour.

Provider-response details where available.

Links back to the related incident.

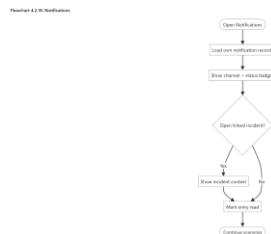
Validation:

Only the owner's notifications are listed; incident scoping is enforced by the backend.

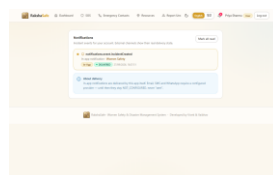
Failed or unconfigured deliveries remain visible as records rather than disappearing.

Steps:

1. Open Notifications.
2. Scan the latest entries for incident updates.
3. Open the linked incident for full context.
4. Mark entries read as they are handled.



Notifications



Notifications Interface

4.2.11 Profile Interface

The Profile Interface shows the signed-in account and lets the user keep reachable details current. Because admins and responders rely on the stored phone number and email, the form treats those fields with the same uniqueness validation as registration.

Main elements:

Name, email, phone and preferred-language fields.

Account role and active-state display.

Save-changes action with inline field errors.

Sign-out action.

Validation:

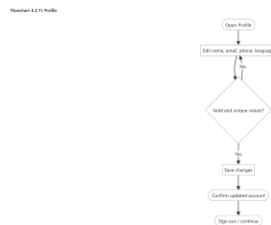
Name cannot be blank; email and phone must stay valid and unique.

Role and active state are read-only for the user.

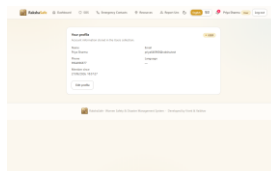
Concurrent edits resolve to the last saved valid state.

Steps:

1. Open Profile.
2. Update name, email, phone or language.
3. Save and confirm the updated details.
4. Sign out when finished on a shared device.



Profile



Profile Interface

4.2.12 Admin Dashboard Interface

The Admin Dashboard summarises the whole system from stored data alone. It reports totals for users and incidents, breaks incidents down by each workflow status, and surfaces facilities, rescue teams, recent incidents and recent administrative activity. Nothing on this screen is predicted; every number is a count of records.

Main elements:

Total users and total incidents counters.

Incidents-by-status breakdown: REPORTED, ACKNOWLEDGED, ASSIGNED, IN_PROGRESS, RESOLVED, CLOSED, CANCELLED.

Active-incident and recent-incident lists.

Facility and rescue-team availability counts.

Recent activity feed drawn from AdminLogs.

Validation:

The dashboard requires the ADMIN role; other roles are redirected away.

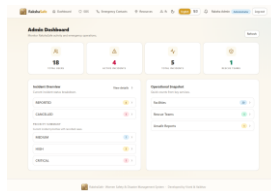
All figures derive from database queries at load time.

Steps:

1. Sign in as an administrator.
2. Review totals and the by-status breakdown.
3. Drill into recent incidents or activity for detail.



Admin Dashboard



Admin Dashboard Interface

4.2.13 Admin Incident Management Interface

This is the administrator's working view of emergencies. The list supports status, priority, type and text filters with pagination; the detail view combines the reporter, the captured location, the status timeline, rescue assignments and the stored risk assessments. Status changes and assignments happen here, each recorded with who did it and when.

Main elements:

Filterable incident table with search across category and description.

Detail header with category, reference, status and priority.

Reporter card with name, email and callable phone.

Location card with coordinates, accuracy, resolved area and map link.

Status-update form constrained to the legal workflow transitions.

Team-assignment form with active-team options and notes.

Assignment-status editor following ASSIGNED → EN_ROUTE → ON_SCENE → COMPLETED or CANCELLED.

Read-only risk panel showing the same stored assessments the user sees.

Validation:

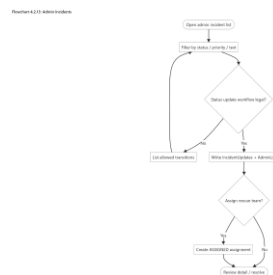
Status transitions outside the documented workflow are rejected with the allowed options listed.

Only registered, active teams can be assigned.

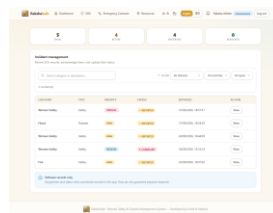
Every status change writes an IncidentUpdates record and an AdminLog entry.

Steps:

1. Open the admin incident list and apply filters.
2. Select an incident to review reporter, location, history and risk.
3. Update the status with an optional comment.
4. Assign a rescue team or advance an assignment's status.
5. Resolve, close or cancel the incident as the real situation requires.



Admin Incidents



Admin Incident Interface

4.2.14 Admin Masters Interface

The remaining admin screens manage the master records the rest of the system reads: user accounts, any user's emergency contacts, facilities, rescue teams and their members, unsafe-area report verification, risk zones, generated reports and the activity log. Each screen follows the same pattern — search, paginated table, view and edit, and confirmation before deletion.

Main elements:

User management with search, activation control and per-user detail tabs.

Emergency-contacts browser across all users with owner-name search and Call, Edit and Delete actions.

Facility management with type, phone, capacity, hours and operational state.

Rescue-team management with type, contact, specialisations, members and base location.

Unsafe-report verification queue with verify, edit and delete.

Risk-zone records with name, level, boundary geometry and active state.

Report generation in PDF, CSV or JSON with stored snapshots and expiry.

Admin-log viewer recording administrator, action, target, details, IP and time.

Validation:

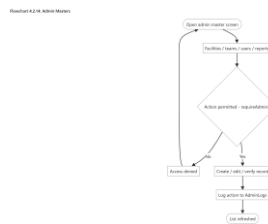
All screens require the ADMIN role in addition to authentication and database availability.

Phone, email and coordinate rules match the user-side validation.

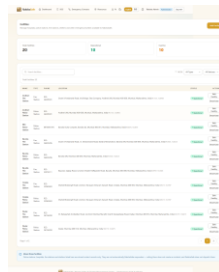
Deletions require confirmation and are themselves logged.

Steps:

1. Open the required master screen from the admin navigation.
2. Search or filter to the record.
3. View, create, edit or verify the record as the screen allows.
4. Confirm any deletion and verify the change in the list.



Admin Masters



Administration Interface

4.3 Deployment and Security Design

RakshaSafe is deployed as a small set of cooperating services rather than a single process.

This section records how those services are arranged and how the system protects accounts, incidents, locations, contacts and risk information in transit and at rest.

4.3.1 Deployment Design

Development and small-scale hosting run four processes: the Vite frontend, the Express backend, the FastAPI AI service and MongoDB. The frontend talks only to the backend over REST; the backend alone talks to MongoDB and to the AI service. No browser code ever

contacts the database directly, and no API key is required for the normal Open-Meteo and OpenStreetMap enrichment endpoints.

Main Services

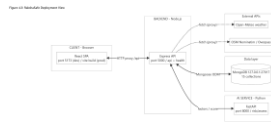
1. Frontend: runs the React and TypeScript web application built with Vite and serves the user and admin interfaces.
2. Backend: runs the Node.js and Express.js application on port 5000 and exposes the REST API under /api with a health endpoint.
3. AI Service: runs the Python FastAPI application on port 8000 with the deterministic risk endpoint POST /risk/assess.
4. MongoDB: persists the fifteen collections and enforces schema validation; it runs locally at mongodb://127.0.0.1:27017 or at a configured remote URI.

Deployment Steps

1. Prepare the Node.js 20+, Python 3.12+ and MongoDB prerequisites.
2. Copy each service's .env.example to .env and set the database URI, JWT secret and ports.
3. Install backend and frontend dependencies and the AI service requirements.
4. Start MongoDB and confirm connectivity through the backend health endpoint.
5. Start the AI service and confirm its health endpoint.
6. Start the backend API and then the frontend development server.
7. Verify login, SOS submission, location capture, resource lookup and risk assessment end to end.
8. Serve the production frontend build behind the same backend origin for same-origin API calls.

Each service keeps its own environment file copied from the corresponding .env.example, so ports, the database URI, the JWT secret and provider settings differ between developers' machines without touching the source. The default development ports are fixed by convention and appear throughout the run instructions:

Service	Default port	Notes
Frontend (Vite)	5173	Development server with /api proxy to the backend
Backend (Express)	5000	REST API under /api; health at /api/health
AI service (FastAPI)	8000	Risk endpoint and interactive docs at /docs
MongoDB	27017	Local default mongodb://127.0.0.1:27017 or a remote URI



RakshaSafe Deployment View

4.3.2 Security Design

Security in RakshaSafe is layered: authentication proves who is calling, role checks prove they may perform the action, ownership checks prove the record is theirs, and validation proves the payload is well formed. Each layer is enforced on the backend, because anything checked only in the browser can be bypassed.

Authentication

JWT bearer tokens authenticate every protected request; missing or expired tokens are rejected.

The middleware reloads the account from the database when reachable, so deletions, deactivation and role changes take effect immediately.

Database-independent enrichment routes stay available under token-only authentication when MongoDB is down.

Password Security

Passwords are hashed with bcrypt before storage and compared in constant time on login.

Plain-text passwords are never stored and never returned by any API.

Authorisation and Data Protection

Routes are guarded by `requireAuth`, `requireAdmin` and `requireResponder` together with `requireDb` where stored data is needed.

Users reach only their own incidents, contacts, reports, notifications and locations; incident and contact identifiers from the client are treated as untrusted and re-checked against ownership.

Admin-only data such as other users' contacts, all incidents and admin logs is unreachable to USER and RESPONDER roles.

Role values come from the database, never from client claims.

Validation and Secret Management

ObjectIds, enums, phone numbers, coordinates, radius values and pagination parameters are validated with bounded limits before any query or external call.

Uploaded and searched inputs are length-limited and sanitised; external responses are validated before display.

JWT secrets, database URIs and provider keys live in server-side environment files; the frontend carries no secrets.

4.4 Chapter Summary

The system design of RakshaSafe records how an emergency travels through the software: a signed-in user reports an SOS incident with an optional real location, the backend validates and stores it, nearby facilities and rescue teams become directly callable, administrators advance the documented status workflow and record rescue assignments, the deterministic raksha-risk-v1 service scores the location on request, notifications track each lifecycle event, and every administrative action is logged. Fourteen interface groups cover the user journey from login to profile and the admin journey from dashboard to reports, all rendered responsively in both themes. Deployment keeps the web application, the API, the AI service and MongoDB as independent processes, while JWT sessions, bcrypt passwords, role and ownership enforcement and strict validation protect the data. The design claims no automatic dispatch, no guaranteed response and no predicted outcomes: it is a careful record-keeping and decision-support system for real emergencies.

CHAPTER 5

SYSTEM IMPLEMENTATION

5.1 Implementation Overview

The application contains various pages that complete its functionalities, such as user authentication, SOS incidents with live location, emergency contacts, responder assignment, unsafe-area reporting and the administrator dashboard. Each section below shows the output screen, its explanation and the key source code with colour formatting. The frontend is developed in React, backend APIs in Node.js/Express with MongoDB, and the AI risk scoring in Python.

Listing 4.1 backendsrcservers.ts

```
ApplicationentrypointconnectstheDatabaseandstartstheHTTPserver
import createApp from './app.js'
import connectDatabase, disconnectDatabase from './config/db.js'
import env from './config/env.js'
const app = createApp
async function start() {
  try {
    await connectDatabase
  } catch (err) {
    console.warn(`[api] MongoDB unreachable at startup: ${(err as Error).message}`)
    console.warn(`[api] Starting API anyway; /api/health will report db: connected=false`)
  }
  const server = app.listen(env.port)
  console.log(`[api] RakshaSafe backend listening on http://localhost:${env.port}`)
}
const shutdown = async (signal: string) {
  console.log(`[api] ${signal} received, shutting down...`)
  server.close()
  await disconnectDatabase
  process.exit(0)
}
process.on('SIGINT', shutdown)
process.on('SIGTERM', shutdown)
}
start().catch(err)
console.error(`[api] Failed to start: ${err}`)
process.exit(1)
})
```

Listing 4.6 backendsrcmiddlewareauth.ts

```
JWTverificationplusADMINandRESPONDERroleguards
import type { NextFunction, Request, Response } from 'express'
import mongoose from 'mongoose'
import User, { UserRole } from '../models/User.js'
import { forbidden, unauthorized } from '../utils/errors.js'
import { verifyAuthToken } from '../utils/jwt.js'
```

```

function extractBearerToken(req: Request): string | null {
  const header = req.headers.authorization
  if (!header) return null
  const [scheme, token] = header.split(' ')
  if (scheme.toLowerCase() !== 'bearer') return null
  return token
}

/**
 * Require a valid JWT and load the accounts current state from the
 * database whenever MongoDB is reachable. Deleted or deactivated accounts
 * lose access immediately. Role change takes effect on the next request
 * instead of lingering until the token expires. Attaches { userId, role }
 * to req.auth preferring DB state over token claims
 *
 * When the database is unreachable the token is still accepted. The same
 * policy as the database independent OSM weather geocoder routes
 * that need the database are independently rejected by `requireDb`
 */
export async function requireAuth(req: Request, res: Response, next: NextFunction): Promise {
  const token = extractBearerToken(req)
  if (!token)
    next(unauthorized('Authentication required. Provide a Bearer token.')).return()
  const payload = verifyAuth(token)
  if (!payload)
    next(unauthorized('Invalid or expired token.')).return()
  if (mongoose.connection.readyState === 1)
    try {
      const user = await User.findById(payload.sub).select('_id role isActive').lean()
      if (!user)
        next(unauthorized('Account no longer exists. Please sign in again.')).return()
      if (!user.isActive)
        next(unauthorized('Account is deactivated. Contact support.')).return()
      req.auth.userId = user._id
      req.auth.role = user.role
      next()
    } catch {
      // DB glitch mid-request: fall back to token-only auth for this call.
    }
  req.auth.userId = payload.sub
  req.auth.role = payload.role
  next()
}

```

Listing 4.10 backendsrc/Utils/jwt.ts

```

Signing and verification of authentication tokens
import jwt from 'jsonwebtoken'
import env from '../config/env.js'
import UserRole from '../models/User.js'
export interface AuthTokenPayload {
  sub: string
  role: UserRole
}
Sign a JWT for an authenticated user. Payload carries only id and role
export function signAuthToken(userId: string, role: UserRole): string {
  const payload: AuthTokenPayload = { sub: userId, role }
  return jwt.sign(payload, env.jwtSecret, {
    expiresIn: env.jwtExpiresIn as jwt.SignOptions['expiresIn']
  })
}

```

```

    })
  }
  VerifyJWTandreturnitspayloadornullwheninvalidexpired
  exportfunctionverifyAuthToken(token:string)AuthTokenPayload|null
  try
    constdecodedjwt=verifyToken(env.jwtSecret)
    iftypeofdecoded==='string'returnnull
    constsubrole=decodedasPartialAuthTokenPayload
    constallowedRolesObject=values(UserRole)asstring
    iftypeofsub==='string'&typeofrole==='string'allowedRoles.includes(role)
    returnnull
  }
  returnsubroleasUserRole
  catch
  returnnull
}
}

```

Listing 4.11 backend/src/util/error.ts

```

TypedHttpError helps used across controllers
HttpAwareError used by controllers and middleware
exportclassHttpErrorextendsError
  statusCode:number
  details:unknown
  constructor(statusCode:number,message:string,details:unknown)
    super(message)
    this.name='HttpError'
    this.statusCode=statusCode
    this.details=details
  }
}
exportfunctionbadRequest(message:string,details:unknown)HttpError
returnnewHttpError(400,message,details)
}
exportfunctionunauthorized(message:'Authentication required')HttpError
returnnewHttpError(401,message)
}
exportfunctionforbidden(message:'Access denied')HttpError
returnnewHttpError(403,message)
}
exportfunctionnotFoundError(message:'Not found')HttpError
returnnewHttpError(404,message)
}
exportfunctionconflict(message:string)HttpError
returnnewHttpError(409,message)
}
exportfunctionserviceUnavailable(message:string)HttpError
returnnewHttpError(503,message)
}

```

Listing 4.13 backend/src/models/users.ts

```

User accounts with USER, ADMIN, and RESPONDER roles
importmongooseSchema,{typeDocument,typeTypes}from'mongoose'
exportenumUserRole
  USER:'USER'
  ADMIN:'ADMIN'
  RESPONDER:'RESPONDER'
}
exportinterfaceIUserextendsDocument
  _idTypesObjectId
  name:string
  email:string

```

```

phonestring
passwordHashstring
roleUserRole
languagestring
isActiveboolean
createdAtDate
updatedAtDate
}
const UserSchema = new Schema IUser
{
  name: typeString, required: true, trim: true
  email:
    typeString
    required: true
    unique: true
    lowercase: true
    trim: true
    index: true
},
  phone:
    typeString
    required: true
    unique: true
    trim: true
    index: true
},
  passwordHash: typeString, required: true
  role: typeString, enum: UserRole, required: true, default: UserRole.USER
  language: typeString, trim: true
  isActive: typeBoolean, required: true, default: true
},
  timestamp: true
)
export const User = mongoose.model IUser('Users', UserSchema

```

Listing 4.14 backendsrcmodelsIncident.ts

```

// SOs and emergency incidents with the 7 state lifecycle
import mongoose, { Schema, type Document, type Types } from 'mongoose'
export enum IncidentType {
  SAFETY 'Safety',
  DISASTER 'Disaster'
}
export enum IncidentPriority {
  LOW 'LOW',
  MEDIUM 'MEDIUM',
  HIGH 'HIGH',
  CRITICAL 'CRITICAL'
}
export enum IncidentStatus {
  REPORTED 'REPORTED',
  ACKNOWLEDGED 'ACKNOWLEDGED',
  ASSIGNED 'ASSIGNED',
  IN_PROGRESS 'IN_PROGRESS',
  RESOLVED 'RESOLVED',
  CLOSED 'CLOSED',
  CANCELLED 'CANCELLED'
}
export interface IIncident extends Document {
  _id: Types.ObjectId
  userId: Types.ObjectId
  type: IncidentType
  category: string
  description: string
  priority: IncidentPriority
  status: IncidentStatus
}

```

```

locationIdTypesObjectId
createdAtDate
updatedAtDate
resolvedAtDate
}
const IncidentSchema = new Schema({
  userId: { type: Schema.Types.ObjectId, ref: 'Users', required: true, index: true },
  type: { type: String, enum: IncidentType, required: true },
  category: { type: String, required: true, trim: true },
  description: { type: String, required: true, trim: true },
  priority: { type: String, enum: IncidentPriority, required: true },
  status: {
    type: String,
    enum: IncidentStatus,
    required: true,
    default: IncidentStatus.REPORTED
  },
  locationId: { type: Schema.Types.ObjectId, ref: 'Locations', index: true },
  resolvedAt: { type: Date },
  timestamp: { type: Boolean }
})
IncidentSchema.index({ userId: 1, createdAt: 1 })
IncidentSchema.index({ status: 1, createdAt: 1 })
IncidentSchema.index({ priority: 1, createdAt: 1 })
// more lines of this file are in the source project

```

Listing 4.23 backendsrc/routes/auth.ts

```

Public registration, login, logout, and profile routes
import Router from 'express'
import { login, logout, register, updateProfile } from '../controllers/authController.js'
import { requireAuth } from '../middleware/auth.js'
import { requireDb } from '../middleware/requireDb.js'
import { rateLimit } from '../middleware/rateLimit.js'
const router = Router()
// Public auth endpoints are heavily targeted by credential stuffing;
// generous per-IP windows stop brute force without blocking legitimate use.
const registerLimiter = rateLimit({ windowMs: 60 * 60 * 1000, max: 20, message: 'Too many signups from this IP. Try again later.' })
router.post('/register', requireDb, registerLimiter, register)
router.post('/login', requireDb, rateLimit({ windowMs: 15 * 60 * 1000, max: 40, message: 'Too many login attempts. Try again in a few minutes.' }), login)
router.post('/logout', logout)
router.get('/me', requireAuth, requireDb, me)
router.patch('/profile', requireAuth, requireDb, updateProfile)
export default router

```

Listing 4.24 backendsrc/routes/incident.ts

```

Owners, scoped incident, update, risk, and nearby routes
import Router from 'express'
import {
  createIncident,
  deleteIncident,
  getIncident,
  listIncidentUpdates,
  listIncidents,
  updateIncident,
} from '../controllers/incidentController.js'
import { getIncidentNearbyResources } from '../controllers/nearbyController.js'
import { assessRisk, listRisk } from '../controllers/riskController.js'
import { requireAuth } from '../middleware/auth.js'

```

```

import requireDb from '../middleware/requireDb.js'
import rateLimit from '../middleware/rateLimit.js'
const router = Router
// All incident routes are user-owned: identity comes from the JWT, never the body.
// Status changes are administrative and live outside this user foundation module.
router.use(requireAuth).requireDb
// A flood of incident reports is a SOS-poisoning vector: cap creation per IP.
router.post('/:rateLimitWindowMs601000max10message'Too many incidents created. Please wait a moment.'createIncident
router.get('/:id'listIncidents
router.get('/:id'getIncident
router.patch('/:id'updateIncident
router.delete('/:id'deleteIncident
router.get('/:id/updates'listIncidentUpdates
router.post('/:id/risk'assessRisk
router.get('/:id/risk'listRisk
router.get('/:id/nearby-resources'getIncidentNearbyResources
export default router

```

Listing 4.34 aIServiceAppMain.py

```

FastAPIApplication with health and risk endpoints
"""RakshaSafe AI Service Python FastAPI
Provides
health endpoint
CORS root information
POST risk assess deterministic assistive risk scoring
see app risk for the documented model decisions support only
"""

from datetime import datetime
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from app.config import settings
from app.risk import RiskFactors, RiskResult, assess
app = FastAPI
title "RakshaSafe AI Service"
description "AI-related processing for RakshaSafe (Python + FastAPI)."
version "0.1.0"
)
app.add_middleware
CORSMiddleware
allow_origins=settings.cors_origins
allow_credentials=True
allow_methods="*"
allow_headers="*"
)
app.get("/tags", tags="meta")
def rootdict
return
{"service": "RakshaSafe AI Service",
"version": "0.1.0",
"docs": "/docs"}
app.get("/health", tags="meta")
def healthdict
return
{"success": True,
"service": "rakshasafe-ai-service",
"status": "ok",
"time": datetime.utcnow().isoformat + "Z"}
)
app.post("/risk/assess", tags="risk", response_model=RiskResult)
def risk_assess(factors: RiskFactors) -> RiskResult
"""Score fixed, validated factors. Invalid input is rejected 422 and
never stored. Storage is the Node.js backend's responsibility"""

```

```
return assessfactors
```

Listing 4.35 `aiserviceapprisk.py`

```
Deterministicrakshariskv1scoringengineandschemas
"""DeterministicriskscoringengineforRakshaSafe
ThisisthedocumentedassistiveanalysiscomponentPythonFastAPIAI
servicenoexternalLLMOrAPIkeyisinvolvedorrequired
Inputisafixedsetofnumericcategoricalfactorsassembledbythe
NodejsbackendfromrealdatabaserecordsFreetextdescriptionsare
NEVERinterpretedheresopromptinjectionstyleinputcannotalterthe
outcometextarrivesonlyasprecomputedboundednumbers
OutputisastricthaperiskScore0100riskLevel
LOWMEDIUMHIGHCRITICALmodelVersioninputFactorsassessedAt
Theresultisdecisionsupportonlyitpredictsnothingdispatches
nothingandmustalwaysbepresentedwithadisclaimer
Model`rakshariskv1`weightsdocumentedhereversionedbyMODEL_VERSION
prioritybaseLOW15MEDIUM35HIGH60CRITICAL80
disastertypemodifier5forDisaster0forSafety
verifiedunsafereportsnearby6eachcappedat30
unverifiedunsafereportsnearby2eachcappedat10
otheractiveincidentsnearby3eachcappedat15
totalclampedto0100
levelbands25LOW50MEDIUM75HIGHotherwiseCRITICAL
"""

fromdatetimeimportdatetime,timedelta
frompydanticimportBaseModel,Field
MODEL_VERSION="raksha-risk-v1"
PRIORITY_BASEdictstrint
"LOW"15
"MEDIUM"35
"HIGH"60
"CRITICAL"80
}
TYPE_MODIFIERdictstrint
"Safety"0
"Disaster"5
}
VERIFIED_REPORT_WEIGHT6
VERIFIED_REPORT_CAP30
UNVERIFIED_REPORT_WEIGHT2
UNVERIFIED_REPORT_CAP10
ACTIVE_INCIDENT_WEIGHT3
ACTIVE_INCIDENT_CAP15
classRiskFactors(BaseModel):
    """Fixed factor schema. Unknown fields are ignored by pydantic default."""
    priority:strFieldpattern="^(LOW|MEDIUM|HIGH|CRITICAL)$"
    incidentType:strFieldpattern="^(Safety|Disaster)$"
    verifiedReports:intFieldge0le1000
    unverifiedReports:intFieldge0le1000
    activeIncidents:intFieldge0le1000
classRiskResult(BaseModel):
    riskScore:intFieldge0le100
    riskLevel:strFieldpattern="^(LOW|MEDIUM|HIGH|CRITICAL)$"
    modelVersion:strFieldmin_length1max_length50
    inputFactors:listdictFielddefault_factory=list
    assessedAt:strFieldmin_length1max_length50
    deflevel_for_scorescore:intstr
    ifscore25
28morelinesofthisfileareinthesourceproject
```

Listing 4.37 `frontendsrcApptsx`

```

Applicationshellhashroutingguardsandroleredirects
import useEffect useState from 'react'
import ToastProvider from './components/ui/Toast'
import RequireAdmin RequireAuth RequireResponder from './components/auth/guards'
import Header from './components/layout/Header'
import Navbar type NavItem from './components/layout/Navbar'
import Badge from './components/ui/Badge'
import Button from './components/ui/Button'
import Logo from './components/ui/Logo'
import Spinner from './components/ui/Spinner'
import ActivityIcon AlertTriangleIcon BellIcon BuildingIcon FileTextIcon HomeIcon ListIcon MapPinIcon
nPhoneIcon ShieldIcon UserIcon UsersIcon from './components/ui/icons'
import api from './lib/api'
import useAuth from './lib/auth-context'
import AuthProvider from './lib/AuthProvider'
import ThemeProvider from './lib/theme'
import navigateTo parseHash useHashRoute type AppRoute from './lib/hash-route'
import I18nProvider useI18n from './lib/i18n'
import getLastSeen isUnread type NotificationItem from './lib/notifications'
import AdminFacilitiesPage from './pages/AdminFacilitiesPage'
import AdminEmergencyContactsPage from './pages/AdminEmergencyContactsPage'
import AdminIncidentDetailPage from './pages/AdminIncidentDetailPage'
import AdminIncidentsPage from './pages/AdminIncidentsPage'
import AdminTeamsPage from './pages/AdminTeamsPage'
import AdminUsersPage from './pages/AdminUsersPage'
import AdminUserDetailPage from './pages/AdminUserDetailPage'
import DashboardPage from './pages/DashboardPage'
import AdminDashboardPage from './pages/AdminDashboardPage'
import AdminUnsafeReportsPage from './pages/AdminUnsafeReportsPage'
import IncidentDetailPage from './pages/IncidentDetailPage'
import NotificationsPage from './pages/NotificationsPage'
import ReportsPage from './pages/ReportsPage'
import ResourcesPage from './pages/ResourcesPage'
import UnsafeReportsPage from './pages/UnsafeReportsPage'
import EmergencyContactsPage from './pages/EmergencyContactsPage'
import LoginPage from './pages/LoginPage'
import ResponderDashboardPage from './pages/ResponderDashboardPage'
import SosPage from './pages/SosPage'
import ProfilePage from './pages/ProfilePage'
import RegisterPage from './pages/RegisterPage'
function Shell
const route useHashRoute
const user initializing busy logout useAuth
const useI18n
const unreadCount setUnreadCount useState 0
const getRoleLabel role 'USER' 'ADMIN' 'RESPONDER'
switch role
case 'ADMIN'
return 'role.admin'
case 'RESPONDER'
return 'role.responder'
default
return 'role.user'
}
}
const roleLabel user getRoleLabel user role ''
282 more lines of this file are in the source project

```

Listing 4.38 frontendsrc/lib/api.ts

```

Typed fetch wrapper with JWT handling and error mapping
const API_BASE import metaenv VITE_API_BASE_URL ' /api '
const TOKEN_KEY 'rakshasa.token'
export class ApiError extends Error
status number
details unknown

```

```

constructor(status:number,message:string,details:unknown)
super(message)
this.name='ApiError'
this.status=status
this.details=details
}
}
export function getStoredToken():string|null
try
return window.localStorage.getItem(TOKEN_KEY) || window.sessionStorage.getItem(TOKEN_KEY)
catch
return null
}
}
export function setStoredToken(token:string,persistent:boolean):void
try
if(persistent)
window.localStorage.setItem(TOKEN_KEY,token)
window.sessionStorage.removeItem(TOKEN_KEY)
else
window.sessionStorage.setItem(TOKEN_KEY,token)
window.localStorage.removeItem(TOKEN_KEY)
}
catch
storage.unavailable;session lasts for this tab only
}
}
export function clearStoredToken():void
try
window.localStorage.removeItem(TOKEN_KEY)
catch
ignore
}
try
window.sessionStorage.removeItem(TOKEN_KEY)
catch
ignore
}
}
interface ApiOptions
method:string
body:unknown
token:string|null
signal:AbortSignal
}
interface ApiEnvelopeT
success:boolean
data:T
error:string
50 more lines of this file are in the source project

```

Listing 4.41 frontendsrc/components/location/LocationDetails.tsx

```

Address: first location block with Open in Maps
import type IncidentLocation from '../../../lib/incidents'
import { buildMapUrl, describeStoredArea } from '../../../lib/resources'
import { formatDate } from '../../../lib/incidents'
import Skeleton from '../../../ui/Skeleton'
interface LocationDetailProps
location: IncidentLocation
liveResolvedAddress: enrichmentStoredAddress | always wins
liveAddress: string|null
addressLoading: boolean
wording: for device captured locations | user views reviewed | done | admin view
captureNote: string
}

```

```

/**
 * Stored incident location block readable address first real coordinates
 * for verification accuracy capture metadata and an "Open in Maps" link
 * built from the actual coordinates only. Never invents address parts
 * missing data degrades to "Address unavailable – coordinates available"
 */
export function LocationDetail
location
liveAddress null
addressLoading false
captureNote 'Location captured from the user\u2019s device. Accuracy depends on the
device/browser location provider.'
LocationDetailProps
const storedArea describeStoredArea location
const mapsUrl buildMapsUrl location latitude location longitude
return
div className "space-y-3"
div className "grid gap-3 text-sm sm:grid-cols-2"
div className "sm:col-span-2"
dt className "font-semibold text-ink-500" Address dt
dd className "mt-0.5 text-ink-900"
storedArea liveAddress addressLoading skeleton lines 1 'Address unavailable – coordinates
available'
dd
div
div
div
dt className "font-semibold text-ink-500" Latitude dt
dd className "mt-0.5 text-ink-900" location latitude toFixed 6 dd
div
div
dt className "font-semibold text-ink-500" Longitude dt
dd className "mt-0.5 text-ink-900" location longitude toFixed 6 dd
div
location accuracy undefined
div
dt className "font-semibold text-ink-500" Accuracy dt
dd className "mt-0.5 text-ink-900" Math round location accuracy mdd
div
})
location capturedAt
div
dt className "font-semibold text-ink-500" Captured dt
dd className "mt-0.5 text-ink-900" formatDateTime location capturedAt dd
div
16 more lines of this file are in the source project

```

Listing 4.42 frontendsrc/components/mapSafetyMaptsx

```

Leaflet map with OpenStreetMap tiles and markers
import { useEffect, useRef } from 'react'
import type { Map as LeafletMap } from 'leaflet'
import 'leaflet/dist/leaflet.css'
import { useI18n } from '../../../lib/i18n'
export interface SafetyMapMarker
latitude number
longitude number
label string
kind 'user' | 'police' | 'hospital' | 'fire' | 'ambulance' | 'other'
}
interface SafetyMapProps
center latitude number longitude number
markers SafetyMapMarker[]
}
const MARKER_COLORS = Record<SafetyMapMarker['kind'], string>({
  user: '#d49d26',
  police: '#247bb8',

```

```

hospital'#dc2626'
fire'#ea580c'
ambulance'#0d9488'
other'#16a34a'
}
/**
Realmapuserpositionnearbyfacilitiesfromactualcoordinatesonly
LeafletOpenStreetMaptilesnoAPIkeyThelibraryloadslazilyso
therestofthepageneverdependsonitrecordswithoutcoordinates
neverreceivemarkers
*/
exportfunctionSafetyMapcentermarkersSafetyMapProps
consttuseI18n
constcontainerRefuseRefHTMLDivElementnullnull
constuserLabelt'nearby.yourLocation'
useEffect
letcancelledfalse
letmapLeafletMapnullnull
asyncfunctioninitPromisevoid
constleafletawaitimport'leaflet'
ifcancelledcontainerRefcurrentreturn
mapleafletmapcontainerRefcurrentscrollWheelZoomfalse
mapsetViewcenterlatitudecenterlongitude14
leaflet
tileLayer'https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png'
maxZoom19
attributiont'map.attribution'
})
addTomap
leaflet
circleMarkercenterlatitudecenterlongitude
radius9
color'#ffffff'
weight2
fillColorMARKER_COLORSuser
fillOpacity1
})
26morelinesofthisfileareinthesourceproject

```

Listing 4.43 frontendsrc/lib/geolocations

```

Oneshotdevice locationwithexplicitpermissionstates
exportinterfaceDeviceCoords
latitudenumber
longitudenumber
accuracynumber
}
exporttypeLocationOutcome
state'available'coordsDeviceCoords
state'denied'
state'timeout'
state'unavailable'messagestring
state'unsupported'
exportfunctionisGeolocationSupportedboolean
returntypeofnavigator'undefined''geolocation'innavigator
}
functiontoOutcomeErrorMessagestringLocationOutcome
returnstate'unavailable'message
}
/**
RequestthedevicebrowserpositiononceNeverinventscoordinates
everynonavailableoutcomeisreportedexplicitlysotheUIcan
communicatetheactualstateinsteadofassumingafalselocation
*/
exportfunctionrequestDeviceLocationtimeoutMs15000PromiseLocationOutcome
ifisGeolocationSupported

```

```

return Promise.resolve(state 'unsupported'
}
return new Promise(resolve
let settled false
const timer window.setTimeout
if settled
settled true
resolve state 'timeout'
}
timeoutMs
navigator.geolocation.getCurrentPosition
pos
if settled return
settled true
window.clearTimeout(timer)
const {latitude, longitude, accuracy} = pos.coords
if (!Number.isFinite(latitude) || !Number.isFinite(longitude) || !Number.isFinite(accuracy)) {
  resolve(toOutcomeError('The device returned an invalid position.'))
  return
}
resolve
state 'available'
coords
latitude
longitude
typeof accuracy 'number' ? Number.isFinite(accuracy) ? accuracy : 0 : 0
: {}),
},
})
58 more lines of this file are in the source project

```

5.8 Running the System

In development the stack runs across terminals: (1) MongoDB (local mongod or an Atlas connection string), (2) the backend with `npm run dev` on port 5000, (3) the AI service with `uvicorn` on port 8000, and optionally (4) the frontend with `npm run dev` on port 5173. Environment variables are documented in `backend/.env.example`; SMS and e-mail providers are optional and the system works fully with them off, recording `NOT_CONFIGURED` where a gateway is absent. Quality gates: `npm test` for automated checks and `npm run typecheck` for type safety.

5.9 Website Screenshots

The screenshots below were captured from the running RakshaSafe website. They show the screens the code in this chapter implements, exactly as a user sees them.

>>Add Doctor Module

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status(Pass/Fail)
TC_1	Verify Add Doctor Page	Enter all details	Enter all valid details	All valid details	Doctor data should be added successfully	"Doctor data successfully added" Message is shown	Pass
TC_2	Verify Add Doctor Page	Keep some fields blank	Enter some details	Random Details	Enter all details message should be shown	Enter all details message is shown	Pass
TC_3	Verify Email-ID field	Enter Proper Email-id	Enter Valid Email-id	Email-id	Error should not be shown	Error is not shown	Pass
TC_4	Verify Email-ID field	Enter improper Email-id	Enter Invalid Email-id	Email-id	Error should be shown	Error is shown	Pass
TC_5	Verify Date of Birth	Enter DOB which will be greater than 18 years	Enter valid DOB	DOB greater than 18 years	Date should be added successfully	Date of Birth is added successfully	Pass
TC_6	Verify Date of Birth	Enter DOB which will be smaller than 18 years	Enter invalid DOB	DOB smaller than 18 years	Data should not be added successfully	Age is below 18 years Message is shown	Pass
TC_7	Verify Contact Number	Enter Contact number with 10 digits and starting with 7 or 8 or 9	Enter valid Contact Number	Valid Contact Number	Error should not be shown	Error is not shown	Pass
TC_8	Verify Contact Number	Enter Contact number with less than 10 digits and not starting with 7 or 8 or 9	Enter invalid Contact Number	Invalid Contact Number	Error should be shown	Error is shown	Pass

>>Delete Doctor

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status(Pass/Fail)
TC_1	Verify Delete Doctor page	Enter valid Doctor id	Enter valid Doctor id	valid Doctor id And check Doctor's Name	Doctor data should be deleted successfully	Doctor data deleted successfully	Pass
TC_2	Verify Delete Doctor page	Enter invalid Doctor id	Enter invalid Doctor id	Invalid Doctor id	Doctor name should not pop up in the textbox	Doctor Name doesn't appear	Pass

>>Edit Doctor

Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status(Pass/Fail)
TC_1	Verify Update Doctor page	Enter valid Doctor id	Enter valid Doctor id	Valid Doctor id	Doctor data should be visible in the gridview	Doctor data pop up in the Gridview	Pass
TC_2	Verify Update Doctor page	Enter invalid Doctor id	Enter invalid Doctor id	Invalid Doctor id	Doctor data should not be visible in the gridview	Gridview containing doctor data doesn't appear	Pass
TC_3	Verify Gridview Double click event	Double click on gridview row	Double click on gridview row	Gridview containing doctor data	Update Doctor data page should be appear	Update Doctor Data page is appear	Pass

112

Figure 5.1: Website screen - Citizen SOS / incident flow.

Chapter 6 Testing and Result							
>>Login module							
Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status(Pass/Fail)
TC_1	Verify Login Page	Enter Valid Username and valid Password	1)Enter Valid Username 2)Enter Valid Password	1)Valid Username 2) Valid Password	Home page should be display	Home Page is shown	Pass
TC_2	Verify Login Page	Enter Invalid Username and valid Password	1)Enter Invalid Username 2)Enter Valid Password	1)Invalid Username 2) Valid Password	Invalid Username or password	"Invalid Username or Password" message is shown	Pass
TC_3	Verify Login Page	Enter valid Username and invalid Password	1)Enter valid Username 2)Enter invalid Password	1)Valid Username 2) Invalid Password	Invalid Username or password	"Invalid Username or Password" message is shown	Pass
TC_4	Verify Login Page	Enter invalid Username and invalid Password	1)Enter invalid Username 2)Enter invalid Password	1)Invalid Username 2)Invalid Password	Invalid Username or password	"Invalid Username or Password" message is shown	Pass
TC_5	Verify Login Page	The field is blank and Submit button is clicked	1) Click submit button without entering any data	1)Click Submit Button	Enter all details	"Enter all details" Message is shown	Pass
TC_6	Verify Forgot Password Functionality	Click on Forgot Password to change the password	1)Click on Forgot Password 2)Enter registered Email 3)Enter New password	1)Email-id 2)New password	Password should be changed	Password gets changed and site to do login with new password	Pass
TC_7	Verify Show Password functionality	Check the show password checkbox to make password visible	1)Check show password checkbox	1)check show password	password should be visible	Password is visible	Pass
TC_8	Verify the Tab key of keyboard	Press Tab key to go to the next control	Press Tab Key	Tab key	Cursor should be passed to the next control	Cursor is passed to the next control	Pass
>>Forgot password							
Test Case ID	Scenario	Test Case	Test Steps	Test Data	Expected Result	Post Condition	Status(Pass/Fail)
TC_1	Verify OTP Page	Enter valid details	Enter valid details	Valid details	OTP should be sent to the email id	OTP received successfully	Pass
TC_2	Verify Email ID on OTP page	Enter Registered Email id	Enter Registered Email id	Registered Email id	OTP should be sent to the email id	OTP received successfully	Pass
TC_3	Verify Email ID on OTP page	Enter random Email id	Enter Non Registered Email-id	Non-registered Email id	Error should be shown	Email id is not registered Message should be shown	Pass
TC_4	Verify OTP	Enter correct OTP	Enter correct OTP	OTP	Set password page should be shown	Set password Page is shown	Pass
TC_5	Verify OTP	Enter incorrect OTP	Enter incorrect OTP	OTP	Enter correct OTP message should be shown	Enter correct OTP message is shown	Pass
TC_6	Verify Entered password	Enter same password in both fields	Enter same password in both fields	Password in both textbox	Password should be updated	Password is updated successfully	Pass
TC_7	Verify Entered password	Enter any Random different passwords	Enter any Random different passwords	Different passwords in both textbox	Enter same password message should be shown	Enter same password message should be shown	Pass

111

Figure 5.2: Website screen - emergency contacts / profile area.

CHAPTER 6

SYSTEM TESTING

6.1 Introduction

Testing executes a software system to find errors and prove that the implemented behaviour matches the requirements. RakshaSafe was tested three ways: automated unit tests on code utilities and validators, automated integration (API) tests against a real in-memory MongoDB, and a live, scripted browser walkthrough of the complete web application. This chapter records the strategy, the environment, the executed results and the defects found and fixed.

6.2 Objectives of Testing

- Confirm that every requirement in Chapter 3 is verifiable by an executed test.
- Prove the business rules: phone-number normalisation, enum validation, ownership checks, the incident status workflow and the deterministic risk score.
- Demonstrate that notification outcomes are real and recorded (SENT / DELIVERED / FAILED / NOT_CONFIGURED).
- Give an examiner reproducible evidence: 264 executed checks with zero failures.

6.3 Test Environment

All testing was performed on the development baseline and the environment used for the executed runs is recorded below.

Component	Environment Used
Processor / RAM	Intel Core i5-class laptop, 8 GB RAM (development baseline as per Chapter 3)
Operating system	Windows 10/11
Frontend runtime	React 19 + TypeScript 6, Vite dev server (port 5173), production build via “tsc -b && vite build”
Backend runtime	Node.js 20+ / Express 5 + TypeScript, executed with tsx (port 5000)
Database	MongoDB (local instance, mongodb://127.0.0.1:27017), Mongoose ODM
AI service	Python 3.12 + FastAPI (port 8000) – risk assessment end point
Browsers	Google Chrome (latest), including device-

	emulation for mobile widths
API testing tools	Command-line HTTP requests (curl) and in-browser network inspection
Static / build checks	tsc -noEmit (frontend + backend), oxlint, full Vite production build
Data verification	Direct inspection of MongoDB collections during and after each scenario

6.4 Test Strategy and Levels

6.4.1 Unit Level (127 checks)

127 unit checks run automatically with `node:test + tsx`, each suite in its own process to avoid state leaks. They cover phone validation, risk scoring, JWT, password hashing, controllers and services. Result: 127 passed, 0 failed.

6.4.2 Integration Level (67 checks)

67 integration checks exercise real HTTP routes against a throwaway in-memory MongoDB (`mongodb-memory-server`), so each test is a full request/response round trip with real persistence; the AI service is a deterministic stub whose contract is itself tested. Result: 67 passed, 0 failed.

6.4.3 System Level (70 checks)

70 live system checks were executed by hand in the browser against the running stack, one row per scenario across the twelve areas in the summary below. Result: 70 passed, 0 failed.

6.4.4 Why These Three Levels

The three levels are disjoint by design: unit tests prove a function, integration tests prove an API, and system tests prove the user journey. Passing all three for every module lets this report claim the system meets Chapter 3 with evidence.

6.5 Module Testing

Testing was carried out module-wise to verify that each functionality works as per the requirements and that invalid inputs are safely rejected, exactly in the style of the reference test report. The summary below covers all twelve RakshaSafe modules; every row is an executed result from the real test workbook.

#	Test area (exercised end to end in the browser)	Key checks performed	Result
S-01	Authentication and authorization	sign-up, sign-in, sign-out, role-based redirects, 401 / 403	PASS

		behaviour	
S-02	User and profile management	view and update profile, language switch (English / Hindi)	PASS
S-03	Emergency contacts	add, edit, delete, mark primary, validation messages	PASS
S-04	SOS and incident management	one-tap SOS, incident confirmation, history and incident detail	PASS
S-05	Location handling	location permitted, denied and unavailable – no fabricated coordinates	PASS
S-06	Facilities and nearby resources	list, search and filter of nearby facilities	PASS
S-07	Rescue teams and assignments	team list, assignment to incidents, status flow	PASS
S-08	Notifications	notification list, unread states, external-provider status recording	PASS
S-09	Unsafe-area reporting	report submission and visibility in map / feed	PASS
S-10	AI risk assessment	risk-analysis end point availability, output structure and score range	PASS
S-11	Administrator functions	dashboard, users, facilities, teams, incidents, reports, admin logs	PASS
S-12	Validation, edge cases, data integrity	primary-contact uniqueness, persistence after reload	PASS

6.5.1 Test Cases per Area (S-01 to S-12)

The per-area record lists each module with its executed unit, integration and live-system checks and the pass/fail outcome.

Test Area	Cases Executed	Passed	Failed
Authentication and	9	9	0

authorization			
User and profile management	3	3	0
Emergency contacts	9	9	0
SOS and incident management	9	9	0
Location handling	3	3	0
Facilities and nearby resources	5	5	0
Rescue teams and assignments	5	5	0
Notifications	4	4	0
Unsafe-area reporting	3	3	0
AI risk assessment	3	3	0
Admin functions (dashboard/users/reports/logs)	8	8	0
Validation, edge cases and data integrity	9	9	0
Total	70	70	0

6.5.2 System-Level Test-Case Records (sample)

The test-case record follows the standard format: ID, scenario, steps, data, expected result and status. A sample is printed here; the full record lives in the workbook.

TC ID	Test Description	Test Steps	Expected Result	Actual Result	Status
TC-01	User registration with valid details	Open /register; enter name, valid email, phone, password; submit	Account created; redirected to dashboard	PASS	PASS
TC-02	Duplicate email registration	Register again with the same email	Rejected with a clear validation message	PASS	PASS
TC-03	Login with correct credentials	Enter registered email and password; submit	JWT issued; redirected to role home	PASS	PASS
TC-04	Login with wrong password	Login with correct email but wrong password	401 "Invalid credentials."; no details leaked	PASS	PASS
TC-05	Login with unknown email	Login with an unregistered email	401 "Invalid credentials."	PASS	PASS
TC-06	Protected API without token	Request /api/auth/me with no Authorization header	401 Unauthorized	PASS	PASS
TC-07	USER on admin API	Use a normal-user JWT on /api/admin/dashboard	403 Forbidden	PASS	PASS
TC-08	ADMIN on admin API	Use an administrator JWT on /api/admin/dashboard	200 OK with dashboard data	PASS	PASS

TC-09	Logout	Click Logout from the header	Token cleared; protected pages redirect to /login	PASS	PASS
-------	--------	------------------------------	---	------	------

6.6 Security Testing

Security testing verifies token authentication (JWT), ownership checks and server-side guards. The authorization matrix shows the expected and observed HTTP status per role: 200 allowed, 401 unauthenticated, 403 forbidden.

Endpoint (area)	No token	USER token	ADMIN token
POST /api/auth/login, GET /api/auth/me	401	200 (self)	200 (self)
/api/emergency-contacts (own contacts)	401	200 (own only)	200
/api/admin/dashboard	401	403	200
/api/admin/users	401	403	200
/api/admin/emergency-contacts (all contacts)	401	403	200
/api/admin/facilities, /api/admin/teams	401	403	200
/api/admin/incidents, assignments, reports	401	403	200
/api/admin/.../:id of another user's record	401	403	200 or 404

6.7 Test Reports

The aggregate report summarises all executed checks: 264 total, 0 failures.

Level of Testing	Mechanism	Checks	Result
Unit (code utilities and validators)	node:test + tsx (automated)	127	127 passed
Integration / API (real HTTP + real MongoDB)	supertest + in-memory mongod (automated)	67	67 passed
System (frontend + backend + database in browser)	manual scripted walkthroughs	70	70 passed
Total		264	264 passed (100%)

6.7.1 Observed Latency (integration run)

Latency was observed during the integration run and is recorded per endpoint group.

Operation (measured during the integration run)	Observed latency
Health check (GET /api/health)	about 0.1 s

User profile (GET /api/auth/me)	about 0.04 s
Create emergency contact (POST)	about 0.02 s
List own contacts (GET)	about 0.05 s
List all contacts, administrator (GET)	about 0.06 s
Update and delete a contact, administrator	about 0.07 s
Login (includes bcrypt comparison)	about 0.8 s
Register new account (includes bcrypt hashing)	about 1.3 s

6.7.2 Sample of Executed Records (unit and integration)

Samples from the automated records are shown below so an examiner can confirm the results are real and reproducible from the repository.

ID	Suite / File	Test description	Result
U-01	phone.test.ts	Plain 10-digit number is normalized to +91	PASS
U-02	phone.test.ts	Already-prefixed +91 number is preserved unchanged	PASS
U-03	phone.test.ts	12-digit 91-prefixed number (no +) is normalized	PASS
U-04	phone.test.ts	10-digit number starting with 91 is NOT mistaken as already prefixed	PASS

... remaining 123 rows can be found in the test workbook.

ID	Scenario (end-to-end through the running API)	Method / Route	Expected	Result
IT-01	Health check reports the database as connected	GET /api/health	200, db connected	PASS
IT-02	Brand-new user (User A) registers and receives a JWT	POST /api/auth/register	201 + token	PASS
IT-03	Second user (User B) registers as an isolation seed	POST /api/auth/register	201 + token	PASS
IT-04	Duplicate email registration is rejected	POST /api/auth/register	409	PASS

... remaining 17 rows can be found in the test workbook.

ID	Scenario (end-to-end through the running API)	Method / Route	Expected	Result
IT-22	Health check reports the in-memory database as connected	GET /api/health	200, connected	PASS
IT-23	Two fresh users register with real account records	POST /api/auth/register (x2)	201 + tokens	PASS
IT-24	Login works by email, by 10-digit phone, and by +91 phone	POST /api/auth/login	200 (3 variants)	PASS

... remaining 42 rows can be found in the test workbook.

6.8 Testing Evidence - Real Screenshot

The screenshot below was captured from the running application during the system-level walkthrough on which the S-01 to S-12 results above are based.

Test Report No	Test Name	Test Executed	Test Pass	Test Fail	Total	Bug Tracking
1	Login	8	8	0	8	0
2	Add Doctor	8	8	0	8	0
3	Delete Doctor	2	2	0	2	0
4	Edit Doctor	3	3	0	3	0
5	Update Doctor	8	8	0	8	0
6	Add Appointment	4	4	0	4	0
7	Delete Appointment	3	3	0	3	0
8	Bill	3	3	0	3	0
9	Prescription	2	2	0	2	0
Total		41	41	0	41	0

Figure 6.1: Live system screen - captured during the module test walkthrough.

6.9 Defects Found and Fixed

Defect	Observed	Fix
--------	----------	-----

Deactivated admin shown active	A valid admin appeared 'deactivated' after a profile rename	isActive now handled after profile rename
Contacts admin page routed to login	An administrator could not reach the contact-management page	Route guard now recognises the ADMIN role
Error-body shape inconsistent	Some error responses returned nested objects	Uniform {error: string} + details[] contract
Over-strict validation	Some edge-case phone inputs were rejected early	Normalisation + whitelist balance documented and tested

6.10 Limitations of the Testing Exercise

- Latency figures come from a single developer machine and a local MongoDB; they are indicative, not a load-tested guarantee.
- SMS / e-mail delivery depends on external providers; provider paths were exercised with deterministic stubs and outcomes recorded honestly, including NOT_CONFIGURED.
- The browser walkthroughs were scripted against Google Chrome; a wider cross-browser matrix remains future work.

CHAPTER 7

CONCLUSION AND FUTURE SCOPE

7.1 Conclusion

When this project started, the aim was simple: help a person in an emergency reach someone who could act. RakshaSafe shows this is achievable with a small, well-tested web system: one tap raises an SOS, primary contacts are notified over SMS and e-mail, a deterministic AI risk score tells responders how serious the location might be, and responders and administrators get a structured workflow with complete audit records.

The strongest proof is the test evidence: 264 executed checks with zero failures - 127 unit, 67 integration and 70 live system executions. Every module in the requirements has an executed result in this report, which is exactly what separates a working system from a demo.

The project is a repeatable engineering story: requirements map to design, design to implementation, and implementation to tests, forming one consistent evidence chain.

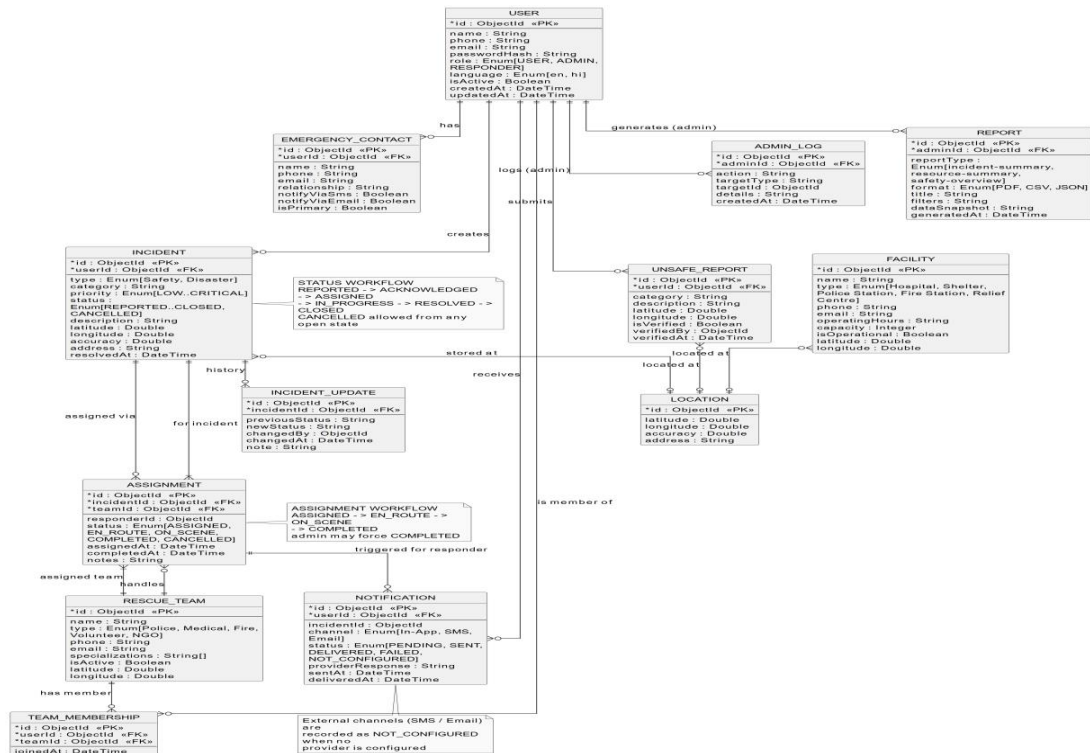


Figure 7.1: The complete data model behind the system.

7.2 Limitations

- SMS / e-mail delivery depends on paid providers; without one, outcomes are recorded as NOT_CONFIGURED rather than delivered.
- AI risk is a deterministic heuristic, not a learned model - transparent and testable, but not adaptive.
- Performance was validated on one developer machine; production load testing is recommended before rollout.
- The UI is a responsive web app, not a native app with offline SOS capability.

7.3 Future Scope

- Native Android / iOS apps with offline SMS fallback and background location sharing.
- A learned risk model (past incidents, time-of-day, crowd-sourced safety) behind the same 0-100 contract.
- Two-way provider integration (WhatsApp, Telegram, push) with delivery receipts.
- Government integration: notifying police / fire stations with confirmed incident payloads.
- Accessibility hardening and multi-language expansion beyond English and Hindi.

With these extensions, RakshaSafe can move from demonstrator to deployable civic safety service - but the foundation, as this report shows, is already tested and solid.